UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A PROJECT REPORT ENTITLED

DYNAMIC CONTEXT-AWARE TRIES FOR KNOWLEDGE GRAPH-CONSTRAINED LARGE LANGUAGE MODEL GENERATION

BY

BERNARD KIRK ADJANOR KATAMANSO

ERICA AMONOR

JOSEPH OSEI NYARKO

JESSICA AFUA ETORNAM NSAFOAH

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

THESIS SUPERVISOR

.....

DR ERIC AFFUM

TARKWA, GHANA

APRIL 2026

DECLARATION

We, BERNARD KIRK ADJANOR KATAMANSO, ERICA AMONOR, JOSEPH OSEI NYARKO and JESSICA AFUA ETORNAM NSAFOAH, declare that this project work is our own original work. It is being submitted for the degree of Bachelor of Science in Computer Science and Engineering at the University of Mines and Technology (UMaT), Tarkwa. It has not been submitted, either in whole or in part, for any degree or examination at any other university.

Signature of Student: _____

BERNARD KIRK ADJANOR KATAMANSO (FOE.41.008.088.22)

Signature of Student: _____

AMONOR ERICA (FOE.41.008.030.22)

Signature of Student: _____

JOSEPH OSEI NYARKO (FOE.41.008.113.22)

Signature of Student: _____

JESSICA AFUA ETORNAM NSAFOAH (FOE.41.008.107.22)

_____ day of _____ (year) _____

ABSTRACT

Though large language models handle language well, they sometimes state incorrect information with high confidence. These false outputs, known as hallucinations, are particularly problematic when answering questions using knowledge graphs, since accuracy requires tracing true connections across structured data. To reduce errors, current techniques such as retrieval-augmented generation link model output to established facts in external sources. Methods like Graph-Constrained Reasoning (GCR) and Decoding on Graphs (DoG) go further by restricting which answers are possible based on predefined graph paths. Yet these systems lock in their constraints before generating a response. Even as the meaning of what is being said evolves during reasoning, the set of allowed paths stays unchanged. Because of this, routes through the graph that are unrelated to the question remain available throughout generation, and the model must sort through them using its own judgment. The same judgment that causes hallucinations in the first place. This dissertation introduces Dynamic Context-Aware Trie (DCA-Trie), a method that adjusts valid knowledge graph paths during decoding rather than fixing them at the start. A sentence transformer scores candidate paths against both the original question and the output generated so far, letting the constraints shift as reasoning progresses rather than holding still from the beginning. One variant applies this filtering when the trie is first built; another updates the constraints step by step during generation. To measure how loosely or tightly a constraint operates (independently of whether the final answer is correct) this work introduces the Semantic Irrelevance Ratio (SIR).

DEDICATION

Dedicated to our parents. For every sacrifice we didn't see and every bit of help we couldn't have asked for. Thank you for everything.

ACKNOWLEDGEMENTS

We thank Dr. Eric Affum for his guidance, patience, and support throughout this project. His supervision was instrumental in shaping the direction of this work. We also express our appreciation to our families, friends, and mentors for their encouragement and understanding throughout the process. Finally, we acknowledge the academic community whose work has informed and inspired this research. This dissertation builds on the contributions of many scholars, and we are grateful for their work.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
BFS	Breadth-First Search
CoT	Chain-of-Thought
CWQ	ComplexWebQuestions
DCA-Trie	Dynamic Context-Aware Trie
DoG	Decoding on Graphs
FNR	False Negative Rate
FSM	Finite State Machine
GCD	Grammar-Constrained Decoding
GCR	Graph-Constrained Reasoning
GNN	Graph Neural Network
GPU	Graphics Processing Unit
KG	Knowledge Graph
KGQA	Knowledge Graph Question Answering
LLM	Large Language Model
NLP	Natural Language Processing
QA	Question Answering
RAG	Retrieval-Augmented Generation
RLHF	Reinforcement Learning from Human Feedback
SBERT	Sentence Bidirectional Encoder Representations from Transformers
SIR	Semantic Irrelevance Ratio
SQL	Structured Query Language
VRAM	Video Random Access Memory

CHAPTER 1

INTRODUCTION

1.1 Background

Large language models (LLMs) have emerged as a cornerstone of modern artificial intelligence, demonstrating remarkable capabilities in natural language understanding, text generation, and multi-step reasoning (Brown *et al.*, 2020). Standard architectures—such as the GPT series and the Llama family (Touvron, Martin, and Stone, 2023)—achieve state-of-the-art performance across diverse tasks by capturing complex statistical patterns from massive text corpora. This pre-training regime enables models to generate coherent responses across wide-ranging domains without relying on explicitly structured knowledge bases.

Despite these advances, LLMs remain highly susceptible to hallucination. The generation of fluent yet factually incorrect or unverifiable text (Ji *et al.*, 2023) can be categorized into intrinsic hallucinations, where the output directly contradicts provided source material, and extrinsic hallucinations, where generated claims cannot be verified against any authoritative source (Huang *et al.*, 2025). This vulnerability is particularly problematic in knowledge-intensive domains. Because autoregressive models generate text sequentially based on prior tokens rather than dynamically verifying facts against external reference points, an initial error can compound across subsequent steps, producing persuasive yet false claims (Ji *et al.*, 2023; Huang *et al.*, 2025).

Knowledge Graph Question Answering (KGQA) is the task of automatically answering natural language questions by querying, traversing, and extracting facts from structured knowledge graphs (KGs). In a KG, facts are organized as interconnected triplets formatted as $(\text{entity}, \text{relation}, \text{entity})$. These enable multi-step reasoning over verifiable real-world knowledge (Bollacker *et al.*, 2008). Standard benchmarks such as WebQSP (Yih et al., 2016) and ComplexWebQuestions (Talmor and Berant, 2018) evaluate this capability via multi-hop path traversal over Freebase. Because answering these queries requires strict adherence to graph topology, unconstrained Large Language Models (LLMs) relying solely on parametric memory frequently generate plausible yet hallucinated reasoning trajectories (He *et al.*, 2021;

Lan *et al.*, 2022).

Retrieval-Augmented Generation (RAG) is widely adopted to mitigate hallucination by retrieving relevant information and supplying them as context during inference (Lewis *et al.*, 2020; Izacard and Grave, 2021). However, because retrieved evidence acts as a soft prompt rather than an absolute boundary, LLMs can still ignore relevant context or generate unsupported claims beyond the retrieved information (Asai *et al.*, 2024).To enforce strict adherence to ground truth, knowledge graph-constrained decoding converts graph topology into hard generation boundaries. By applying a validity mask over the output vocabulary at each step, a constraint oracle guarantees that every generated token corresponds to a valid path in the KG (Willard and Louf, 2023; Geng *et al.*, 2023) Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025) operationalizes this paradigm using a pre-computed KG-Trie as the constraint oracle during beam search. While GCR guarantees structural faithfulness, its static offline trie is constructed before decoding begins. Consequently, the oracle cannot account for the semantic intent of the query or the evolving context of the partial reasoning chain, forcing the LLM to rely on internal parametric knowledge to filter out structurally valid yet semantically irrelevant paths—defeating the primary motivation for constrained generation (Li *et al.*, 2025; Luo *et al.*, 2025) (Li et al., 2025; Luo et al., 2025).Decoding on Graphs (DoG) (Luo *et al.*, 2025) attempts to resolve this rigidity by dynamically expanding the local constraint trie during decoding. However, DoG selects candidate nodes based purely on graph connectivity. Because it admits all topographically adjacent nodes without semantic filtering, the model incurs unnecessary computational overhead while remaining vulnerable to semantically off-target path traversal.

These limitations motivate the Dynamic Context-Aware Trie (DCA-Trie) framework proposed in this project. DCA-Trie extends existing KG-constrained decoding by incorporating semantic relevance directly into the constraint oracle, conditioning the valid token set on the knowledge graph structure, the input question, and the partially generated reasoning chain. By combining structural constraints with dynamic semantic filtering, the proposed framework aims to reduce constraint permissiveness, improve answer accuracy, and maintain the structural faithfulness guarantees provided by existing constrained decoding methods.

1.2 Objectives of Thesis

Aim

To design, implement, and evaluate a Dynamic Context-Aware Trie (DCA-Trie) framework for knowledge graph-constrained LLM generation, conditioning each decoding step on the knowledge graph, the input question, and the partial reasoning chain to improve multi-hop KGQA accuracy. The framework must preserve 100% structural faithfulness throughout.

Objectives

To achieve this aim, the following objectives have been established:

i. To characterize the constraint permissiveness problem in existing KG-constrained decoding frameworks by defining and measuring the Semantic Irrelevance Ratio (SIR) of GCR's static KG-Tries on WebQSP, stratified by hop depth.

ii. To design a symbolic constraint oracle (TypeOracle) that filters candidate KG paths using two ontology-based gates, a range gate checking property ranges at each hop and a type gate checking terminal entity types at the final hop, eliminating the encoder cost and threshold sensitivity of embedding-based approaches.

iii. To implement DCA-Trie v1, a static filtering variant that applies the TypeOracle at KG-Trie construction time, reducing trie size and keeping the false negative rate below 5% on gold paths on the WebQSP validation set.

iv. To implement DCA-Trie v2, a dynamic variant in which the trie is updated after each committed triplet using the TypeOracle conditioned on the question and partial generation $y^{<t}$, directly operationalizing Eq. **??**.

v. To evaluate DCA-Trie v1 and v2 against the GCR baseline and an unconstrained chain-of-thought (CoT) baseline on WebQSP, measuring Hits@1, F1, structural faithfulness, SIR, and average trie size, with results stratified by question hop depth.

1.3 Tools and Facilities Used

1.3.1 Software Tools

Table 1.1 presents the software tools used in this project.

Table 1.1 Implemented Software Tools

Tool	Use in Project	Rationale
Python / HuggingFace Transformers	Model inference pipeline; loading and running the Llama-3.1-8B checkpoint; constrained decoding callback	Enables direct use of the released GCR checkpoint with standard tokenization and generation utilities.
GCR Framework	Primary baseline; KG-Trie construction, graph-constrained decoding, and result reproduction on WebQSP and CWQ	Provides the trie-based constrained decoding mechanism that DCA-Trie extends.
Sentence-transformers / all-MiniLM-L6-v2	Semantic relevance scoring in early design iterations; later replaced by the symbolic TypeOracle	Used in the cosine-similarity and decomposed-score approaches before the final symbolic design.
PyTorch	Deep learning framework underpinning model inference and custom decoding logic	Supports SDPA attention and bfloat16 inference via HuggingFace.
marisa_trie	KG-Trie construction and prefix-based token lookup for constrained decoding	Provides $O(1)$ prefix lookup for the valid-token mask at each decoding step.
NetworkX	Graph construction and depth-limited DFS path enumeration from question entities	Builds directed graphs from Freebase triplets and enumerates candidate reasoning paths.
Freebase / WebQSP and CWQ Datasets	Knowledge graph source and KGQA benchmarks for evaluation	Standard benchmarks enabling direct comparison with prior work.
NVIDIA GeForce RTX 4090 (24 GB)	GPU for all inference experiments; Llama-3.1-8B constrained decoding with beam search	Ada Lovelace architecture with sufficient VRAM for bfloat16 inference of the 8B-parameter model.
Visual Studio Code + Git / GitHub	Development environment and version control	Supports coding, version tracking, and reproducibility.

1.3.2 Facilities

The following facilities were utilized in the execution of this project:

i. NVIDIA GeForce RTX 4090 GPU (24 GB VRAM) for all model inference and constrained decoding experiments.

ii. University library and online academic databases (ACM Digital Library, arXiv, Google Scholar, ACL Anthology) for literature review and citation verification.

iii. Shared team repository hosted on GitHub for version control, experiment logging, and reproducibility of all reported results.

1.4 Methods Used

The project employs the following methods:

i. Systematic review and critical analysis of the academic literature on constrained decoding, knowledge graph question answering, hallucination in LLMs, and retrieval-augmented generation.

ii. Formal gap analysis of the four foundational papers (GCR (Luo *et al.*, 2025), DoG (Li *et al.*, 2025), GCD (Geng *et al.*, 2023), and Outlines (Willard and Louf, 2023)) to identify the permissiveness problem and the absence of generation-state feedback in the constraint oracle.

iii. Formal specification of the DCA-Trie framework, including the SIR metric, the relevance scoring mechanism, the static filtering procedure for v1, and the step-wise dynamic expansion rule for v2.

iv. Reproduction of the GCR baseline on WebQSP using GCR's publicly released Llama-3.1-8B checkpoint to establish a verified starting point within 1–2% of published Hits@1.

v. Quantitative evaluation of all configurations on WebQSP and CWQ using Hits@1, F1, faithfulness rate, SIR, and average trie size per step, stratified by question hop depth following the protocol of Lan *et al.* (2022).

1.5 Scope of Work

This project targets the permissiveness problem in GCR and DoG: the fact that existing constraint oracles admit every structurally reachable path, regardless of whether the path is relevant to the question being asked. The evaluation is restricted to multi-hop KGQA over Freebase-grounded benchmarks (WebQSP and CWQ), the standard settings in this literature. All experiments use GCR's publicly released Llama-3.1-8B checkpoint (Luo *et al.*, 2025); no model training or fine-tuning is performed. The semantic relevance scorer uses all-MiniLM-L6-v2 without domain-specific adaptation.

Generalization to knowledge graphs other than Freebase, or to non-KGQA tasks such as entity linking, would answer a different question: whether DCA-Trie transfers across domains. This is identified as future work. Formal proofs of the faithfulness guarantee under dynamic pruning are beyond scope; empirical measurement of the gold answer false negative rate provides the relevant evidence. Production-scale deployment and latency optimization are engineering extensions, not research contributions.

The primary contribution is the DCA-Trie framework and the Semantic Irrelevance Ratio metric. Secondary contributions are the empirical characterization of constraint permissiveness in existing frameworks and the ablation analysis isolating semantic filtering from dynamic expansion. All code, configurations, and curated evaluation datasets will be made publicly available.

1.6 Organization of Work

Chapter 1 identifies the problem and motivates DCA-Trie. Chapter 2 reviews literature on hallucination, KGQA, constrained decoding, and related frameworks, establishing the gaps that motivate context-aware constraint mechanisms. Chapter 3 presents the DCA-Trie formulation, research questions, evaluation criteria, and implementation of v1 and v2 within the GCR pipeline (Luo *et al.*, 2025). Chapter 4 reports experimental results on WebQSP and CWQ, analyzing structural faithfulness, semantic irrelevance, answer accuracy, and findings from ablation and error analysis. Chapter 5 summarizes contributions, discusses practical implications, and identifies directions for future work.

CHAPTER 2

LITERATURE REVIEW

While large language models (LLMs) are good at many tasks, their use of autoregressive generation makes them likely to produce fluent but incorrect information (Ji *et al.*, 2023). For knowledge-intensive question answering, this unreliability is a major roadblock: the ability to verify reasoning against external facts is essential. To tackle this issue, recent studies have focused on grounding models in structured external facts through knowledge graphs. Still, keeping a model strictly aligned with these graphs during generation, without losing its natural flow, is a difficult challenge.

This literature review looks at the current research on these challenges and examines how knowledge graph constraints might help ensure factual accuracy during multi-hop reasoning. Instead of immediately discussing the proposed solution, this chapter outlines the connections between language model hallucination, multi-step validation, and structured grounding. By assessing current strategies to reduce errors, from improved prompting to fixed decoding, this chapter shows where soft-grounding methods do not meet expectations, highlighting the specific technical gaps this project addresses.

2.1 Large Language Models and the Hallucination Problem

Modern LLMs are built on the Transformer architecture (Vaswani *et al.*, 2017), using scaled dot-product self-attention to weight token importance regardless of positional distance. The field has largely moved toward decoder-only models with causal masking, where each token attends only to preceding tokens. The Llama-3.1-8B model (Dubey *et al.*, 2024) is a prominent example of this design and has been adopted as the backbone in recent KG-constrained decoding work (Luo *et al.*, 2025).

Text production in these models is fundamentally autoregressive: given a vocabulary $\mathcal{V}$, an input x, and prior tokens $y_{<t}$, the model computes $P(y_t|y_{<t}, x)$ over the full vocabulary. Since this distribution is always defined over every token, any token has a positive, non-zero chance of being generated at any step. Decoding strategies such as beam search and nucleus

sampling (Holtzman *et al.*, 2020) determine how a token is chosen from this distribution. This autoregressive nature creates fluent, coherent text, but since each step relies on prior outputs without an external verifier, it also directly causes hallucination.

LLMs are pretrained on vast unlabelled text using next-token prediction. The GPT series (Brown *et al.* 2020; OpenAI 2024) and the Llama series (Touvron, Martin, and Stone 2023; Dubey *et al.* 2024) show that scaling this simple task leads to powerful downstream capabilities. During pretraining, the model internalizes patterns and facts from its training data, but this memory is fundamentally static and statistical. The model lacks a way to interact with dynamic external facts or ensure that its outputs reflect verified truths.

2.1.1 Definition and Structural Causes

Hallucination in natural language generation refers to the production of content that is fluent and internally coherent yet factually unsupported by or in contradiction with verifiable sources (Ji *et al.*, 2023). Huang *et al.* (2025) refine this into *intrinsic hallucination* (output contradicts provided source context) and *extrinsic hallucination* (output contains claims unverifiable against any source). Both types appear at non-trivial rates: Ji *et al.* (2023) survey over 30 works finding hallucination rates from 15% to over 60%, and Luo *et al.* (2025) report that retrieval-augmented reasoning approaches produce hallucinated reasoning steps in approximately one-third of cases even with direct access to a knowledge graph.

The primary cause is structural. Three aspects of the autoregressive mechanism make hallucination an ongoing risk. First, the generation distribution is based entirely on learned parameters, with no component that verifies tokens against an external source. Second, errors propagate forward: a hallucinated token at step t becomes conditioning context for step $t+1$, producing fluent continuations of an incorrect chain. Third, the distribution is defined over the full vocabulary at every step, meaning any token has a positive chance of selection regardless of factual correctness (Brown *et al.*, 2020; Huang *et al.*, 2025; Ji *et al.*, 2023).

2.1.2 The Scaling Paradox

A common argument is that larger models should eventually achieve factual reliability. Evidence does not support this. Lin, Hilton, and Evans (2022) introduce TruthfulQA and find that larger models do not outperform smaller ones in truthfulness and in some categories

show *inverse scaling*—they reproduce widespread incorrect beliefs more effectively. Perez *et al.* (2023) show that RLHF-trained models generate confident-sounding responses aligned with apparent user expectations rather than verifiable facts. Kandpal *et al.* (2023) demonstrate that larger models improve on frequently-seen entity combinations but not on compositional questions requiring unseen fact combinations—precisely the structure of multi-hop KG questions. Mallen *et al.* (2023) confirm significant hallucination rates for GPT-4 on tail-entity questions regardless of model size. Together, these findings establish hallucination as a structural feature of the autoregressive mechanism, not a scaling issue.

2.2 Chain-of-Thought and Its Limits

2.2.1 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting (Bosma *et al.*, 2022) is the dominant paradigm for eliciting multi-step reasoning from LLMs. By including worked examples that show intermediate reasoning steps, the model learns to decompose complex problems into simpler sub-problems. Bosma *et al.* (2022) demonstrate substantial accuracy improvements on arithmetic, commonsense, and multi-hop QA benchmarks, with the capability emerging at approximately 100 billion parameters. Zero-shot chain-of-thought (Kojima *et al.*, 2022) achieves similar benefits by adding "Let's think step by step" to the prompt.

Despite these improvements, CoT has a fundamental structural limitation: each intermediate step is generated by the same autoregressive mechanism. An incorrect step propagates forward as conditioning context, and no mechanism verifies that any step corresponds to a fact in an external source. A CoT chain can be entirely fluent and internally coherent while containing factually fabricated steps.

2.2.2 Extensions: Self-Consistency and Tree-of-Thought

Self-consistency (Wang *et al.*, 2023) samples several independent CoT chains and chooses the final answer by majority vote, reducing sensitivity to individual hallucinated chains. However, a majority of independently hallucinated chains can still produce an incorrect answer, especially on deep multi-hop questions. Tree-of-Thought (Cao *et al.*, 2023) treats generation as search through a tree of partial reasoning sequences, using the LLM as both

generator and evaluator. The key limitation persists: pruning decisions rely on the model's own parameters without external verification of factual accuracy.

2.2.3 The Faithfulness Ceiling

All prompting-based methods—few-shot, CoT, self-consistency, tree-of-thought—modify the *input* to the LLM without modifying the *generation mechanism*. Since decoding happens over the full vocabulary, each token has a strictly positive probability at every step. True structural faithfulness requires that invalid tokens receive exactly zero probability. This can only be achieved by directly changing the decoding algorithm (Geng *et al.*, 2023; Willard and Louf, 2023).

2.3 Knowledge Graphs and Knowledge Graph Question Answering

2.3.1 Knowledge Graphs

A knowledge graph is a structured database of verified facts, represented as directed, labelled triplets (e, r, e') where $e, e' \in \mathcal{E}$ are entities and $r \in \mathcal{R}$ is a relation type. Figure 2.1 illustrates a simplified fragment of such a structure. Freebase (Bollacker *et al.*, 2008), the KG underlying the evaluation benchmarks used in this work, encodes tens of millions of triplets across geography, history, entertainment, and science (Yih *et al.*, 2016; Talmor and Berant, 2018).

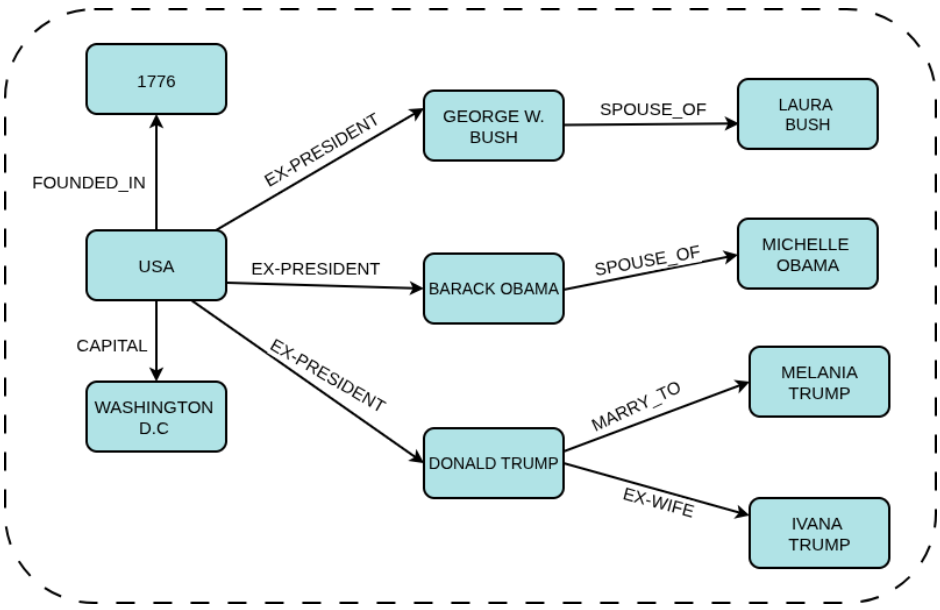

Figure 2.1 Knowledge Graph Fragment

The critical distinction between a KG and an unstructured text corpus is that every triplet has been explicitly verified and is machine-readable in deterministic form. This makes KGs suitable as sources for hard constraints on generation: a candidate token can be verified via exact lookup rather than probabilistic semantic matching. A multi-hop reasoning path of length L is a sequence $(e_0, r_1, e_1, \ldots, r_L, e_L)$ in which each consecutive triplet (e_{i-1}, r_i, e_i) is a verified member of $\mathcal{T}$ (Lan *et al.*, 2022).

2.3.2 Knowledge Graph Question Answering

KGQA requires a system to identify the answer entity e_L to a natural language question q by reasoning through a chain of verified KG triplets. Formally: given question q with question entities $\mathcal{E}_q \subseteq \mathcal{E}$ identified by entity linking, identify the terminal entity of the correct reasoning path in $\mathcal{G}$ (Lan *et al.*, 2022). Early approaches relied on semantic parsing or information retrieval over KG subgraphs. More recent approaches use LLMs as retrievers, agents iteratively querying the KG, or generators conditioned on retrieved subgraph context (Jiang *et al.*, 2023a; Jin *et al.*, 2024; Luo *et al.*, 2024). The move toward LLM-based KGQA has improved fluency but brought back hallucination risks.

2.3.3 Evaluation Benchmarks

WebQSP (Yih *et al.*, 2016) contains 4,737 questions on Freebase with reasoning depths of one to two hops, evaluated using Hits@1 and F1. ComplexWebQuestions (CWQ; Talmor and Berant 2018) builds on WebQSP through conjunction, composition, and comparative operations, resulting in about 34,000 questions requiring up to four hops. WebQSP illustrates a case where valid paths are relatively few; CWQ shows a case with many valid paths, making correct path selection harder.

2.3.4 Multi-Hop Complexity

For an entity with average out-degree d in the KG, the number of valid paths of length L from the question entities increases as $|\mathcal{E}_q| \times d^L$. For well-connected Freebase entities where $d \approx 20$ and a question may have up to three question entities, a three-hop question can generate up to 24,000 structurally valid paths, but only one leads to the correct answer (He *et al.*, 2021; Lan *et al.*, 2022). At each step, the model must choose the correct next token from a vast array of valid yet incorrect options.

2.4 Constrained Decoding for Faithful Text Generation

Constrained decoding restricts generation to a predefined valid set at each step by directly intervening in the decoding mechanism. Unlike prompting, which changes the input to the LLM, constrained decoding alters the token selection process itself. Invalid tokens receive zero probability before sampling, making their generation structurally impossible.

2.4.1 Logit Masking

The main mechanism is logit masking (Geng *et al.*, 2023; Willard and Louf, 2023), which computes a binary mask over the vocabulary at each generation step, setting the log-probability of invalid tokens to $-\infty$ before the softmax. This forces their selection probability to exactly zero, making invalid tokens structurally impossible to generate.

2.4.2 Grammar- and Format-Constrained Decoding

Geng *et al.* (2023) introduce Grammar-Constrained Decoding (GCD), constraining generation to strings conforming to a context-free grammar via an Earley parser oracle. Willard and Louf (2023) reduce the computational cost with an efficient finite-state machine (FSM) formulation, precomputing an index from automaton states to valid vocabulary subsets for amortised $O(1)$ lookup. De Cao *et al.* (2021) introduce autoregressive entity retrieval (GENRE), limiting generation to entity names via a trie of known surface forms.

Additional constrained decoding methods include Synchromesh (Poesia *et al.*, 2022) for program synthesis, PICARD (Scholak, Schucher, and Bahdanau, 2021) for schema-constrained SQL parsing, NeuroLogic Decoding (Lu *et al.*, 2021) for propositional logic constraints over lexical items, and grid beam search (Hokamp and Liu, 2017) for complex lexical constraint sets. Zhang *et al.* (2023) analyse computational tractability under various constraint classes, confirming that finite-state constraints admit efficient enforcement via precomputed indices.

The shared limitation of all these approaches is that they enforce output *format* rather than *factual content*. A grammatically valid output can contain entirely hallucinated facts. The extension from format constraints to KG path constraints requires a constraint oracle that encodes the relational structure of the knowledge graph itself.

2.4.3 Knowledge Graph-Constrained Decoding

Applying constrained decoding to knowledge graph faithfulness uses the KG itself as the constraint oracle. At each generation step, only tokens corresponding to valid continuations of a KG reasoning path are allowed, making hallucination of KG facts structurally impossible.

Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025) builds a KG-Trie (a prefix tree of all KG paths within L hops of the question entities, created by BFS before generation) and uses it as the constraint oracle during beam-search decoding. GCR achieves 92.6 Hits@1 on WebQSP and 75.8 on CWQ with 100% structural faithfulness, using a Llama-3.1-8B backbone. The trie is created once and remains unchanged: the valid token set is fixed regardless of what has been generated.

Decoding on Graphs (DoG) (Li *et al.*, 2025) presents step-wise trie expansion as a partial solution to GCR's static oracle. After locking in an entity, the constraint trie is expanded with all KG neighbors of that entity, making the oracle topologically dynamic. DoG achieves 90.99 Hits@1 on WebQSP and 76.08 on CWQ with 100% structural faithfulness. However, the expansion rule includes all KG neighbors regardless of relevance to the question, and step-wise trie construction compromises the efficiency of precomputed structures.

RouterKGQA (Yuan *et al.*, 2026) routes between a specialized KG-reasoning model and a general LLM based on question complexity. It uses SBERT embeddings (`nomic-embed-text-v1`) to filter candidate answers after generation. RouterKGQA's semantic scoring operates at the answer selection level rather than at the token constraint level: paths are created first and filtered afterward.

2.4.4 Beam Search and Trie-Based Constraints

Beam Search. First introduced in the HARPY system (Lowerre, 1976), beam search keeps a set of the k most promising partial hypotheses at all times. At each step, all k hypotheses are expanded, ranked by total log-probability, and reduced to the top k (Holtzman *et al.*, 2020). This reduces early-error multiplication while remaining computationally viable.

The Prefix Trie. A trie (Fredkin, 1960) is a tree-based data structure where a node's position indicates the prefix it represents and branches show valid next tokens. In constrained decoding,

a pre-computed trie yields the valid next-token subset in $O(1)$ time, making it the most scalable option for factual constraints at inference speed (Willard and Louf, 2023).

The KG-Trie. GCR's KG-Trie (Luo *et al.*, 2025) serializes verified multi-hop paths from the KG into string format mapped to the LLM's token vocabulary. Each edge represents a specific valid token rather than a single character.

When beam search is merged with a KG-Trie oracle as visualized in Figure 2.2, the decoding process transforms into a strictly bounded traversal of the prefix tree.

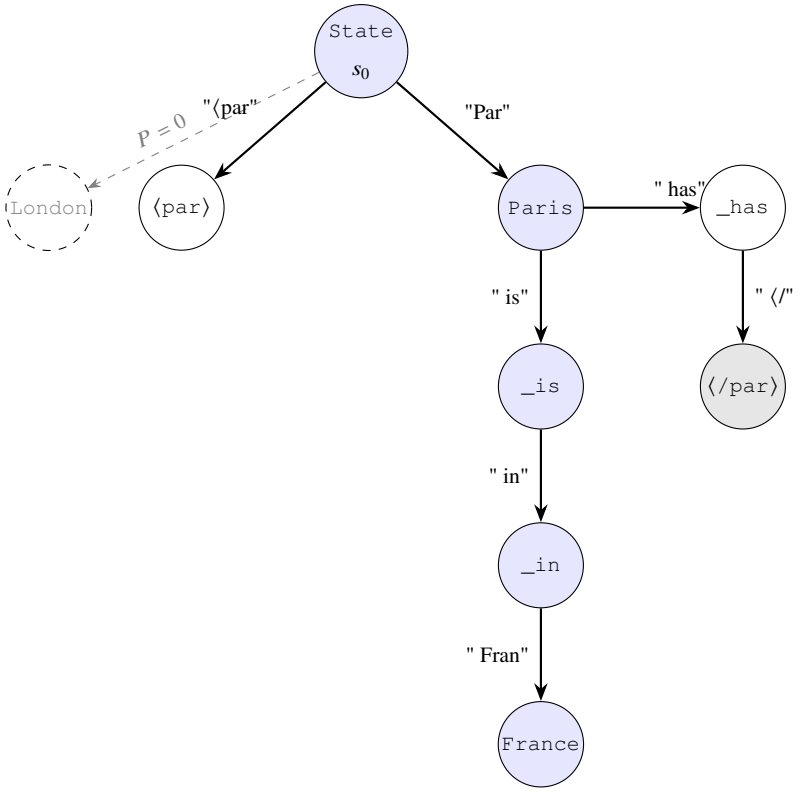

Figure 2.2 KG-Trie Constrained Generation

At each autoregressive step, the current beam hypotheses map to specific states in the KG-Trie. The language model calculates a probability distribution over its entire vocabulary, but a mathematical mask forces the probability of any candidate token not validly present as a child node in the trie to exactly zero ($-\infty$ prior to softmax normalization). The beam search then advances strictly down the permissible branches, restricting the LLM's autoregressive generation to pre-verified deterministic paths while still allowing the model to express internal preference among factually valid chains.

2.5 Retrieval and Soft Grounding

Retrieval-augmented generation (RAG) (Lewis *et al.*, 2020) is the leading approach for grounding LLM outputs in external knowledge. A dense retriever selects relevant passages based on embedding similarity, and a reader LLM generates the answer from the retrieved context. Fusion-in-Decoder (Izacard and Grave, 2021) enhances this by encoding multiple passages separately and merging them in the decoder. Self-RAG (Asai *et al.*, 2024) adds reflection tokens for when to retrieve and whether passages support claims. FLARE (Jiang *et al.*, 2023b) triggers retrieval when next-token confidence drops. IRCoT (Trivedi *et al.*, 2023) interleaves retrieval with CoT steps, improving multi-hop accuracy. GraphRAG (Edge *et al.*, 2024) builds community-structured KGs from document collections for complex relational queries.

In the KGQA context, several approaches enhance RAG with structured context. He *et al.* (2021) retrieve KG subgraphs with attention-guided traversal. Yasunaga *et al.* (2021) combine text passages and KG neighborhoods through graph neural networks. Shi *et al.* (2021) retrieve KG paths as natural language context strings. Sentence-BERT (Reimers and Gurevych, 2019) encoders produce dense vector representations where cosine similarity proxies semantic alignment, applied in these systems to rank candidate KG paths as soft retrieval signals (Shi *et al.*, 2021; Saxena, Chakrabarti, and Talukdar, 2022).

2.5.1 The Structural Limitation of Soft Grounding

Despite their empirical improvements, all RAG approaches share a structural limitation: retrieved context is a soft influence on generation. As shown in Figure 2.3, the retriever determines what the LLM sees; it does not determine what the LLM is permitted to generate. Because the generation mechanism remains unchanged, there exists a nonzero probability of generating any token at any step, including tokens that correspond to no verified KG fact. Structural faithfulness with probability one cannot be achieved under any architecture that modifies only the input context without modifying the token selection mechanism (Luo *et al.*, 2025; Geng *et al.*, 2023).

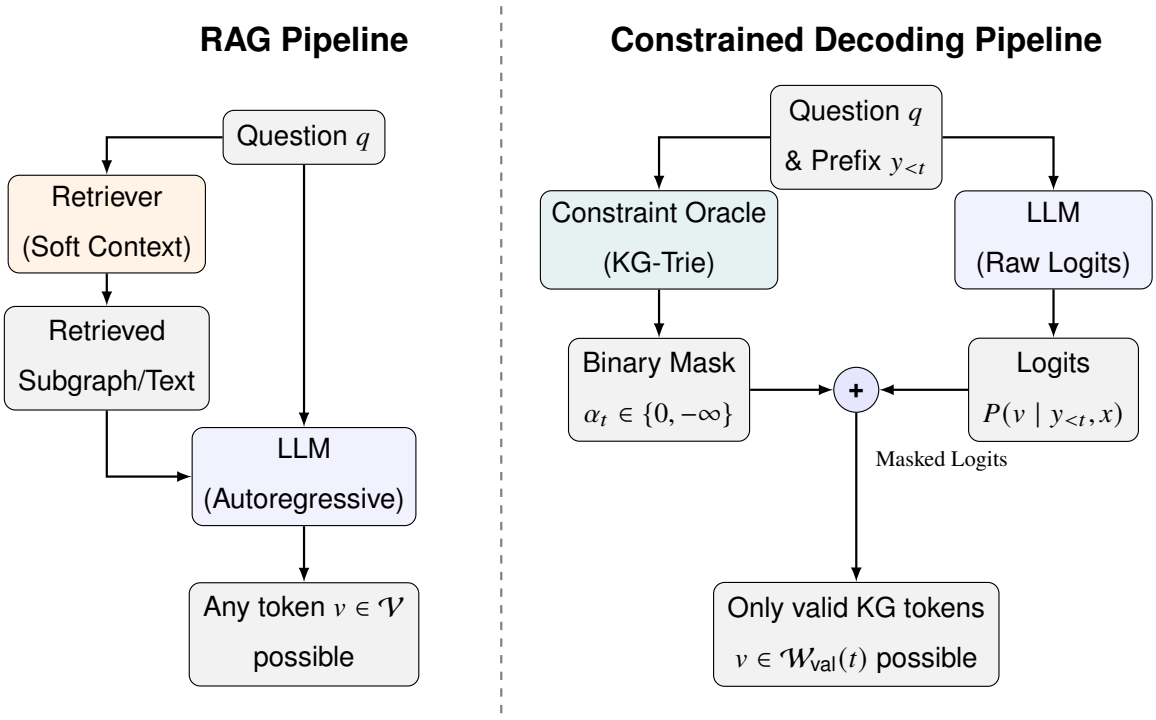

Figure 2.3 Standard RAG vs. KG-Constrained Decoding

2.6 Synthesis: Three Unresolved Gaps

The review shows that two different research paths, constrained decoding for structural faithfulness and semantic similarity for relevance-guided reasoning, have developed mainly in parallel without being combined at the constraint-oracle level. Constrained decoding methods (Luo *et al.*, 2025; Li *et al.*, 2025) achieve structural faithfulness but create their constraint oracles without considering question semantics or the context of generation. This leads to valid token sets that include many irrelevant paths. Semantic similarity methods (Shi *et al.*, 2021; Saxena, Chakrabarti, and Talukdar, 2022; Yuan *et al.*, 2026) improve relevance but act as soft signals that affect retrieval or selection after the fact, without changing the generation process.

Three specific gaps emerge from this review.

Gap 1: The Permissiveness Problem. Current KG-constrained decoding frameworks build their constraint oracles based on graph structure and question entities only. They do not consider question semantics or the current generation state. In the case of multi-hop questions, this results in valid token sets with thousands of structurally correct but semantically irrelevant paths at each generation step. The LLM has to sort through these paths using its own

knowledge, which reintroduces the risk of hallucination that constrained decoding aimed to eliminate (Luo *et al.*, 2025; Li *et al.*, 2025).

Gap 2: The Static-Dynamic Efficiency Tradeoff. GCR's precomputed static trie is efficient in computing but lacks semantic awareness and remains the same at every generation step. DoG's step-wise topological expansion adds some dynamism but requires reconstructing the trie at every step and does not include semantic filtering (Li *et al.*, 2025; Willard and Louf, 2023). No current framework achieves semantic responsiveness, where the valid set narrows progressively as reasoning moves in a specific direction, while also maintaining the efficiency of precomputed constraint structures.

Gap 3: The Absence of a Constraint Quality Metric. No existing work offers a metric that measures constraint permissiveness separately from the accuracy of downstream answers. Without this metric, it is not possible to track progress on reducing irrelevant paths in the valid set or to compare frameworks based on constraint quality without factoring in model quality. These three gaps highlight a key challenge at the intersection of the two research paths discussed in this chapter. This challenge involves integrating semantic relevance scoring directly into the constraint oracle and conditioning the valid token set based on the knowledge graph, question semantics, and the evolving state of generation.

Table 2.1 summarizes the key works reviewed in this chapter, categorizing them by their approach, whether they provide structural faithfulness guarantees, and whether they incorporate semantic conditioning into their constraint mechanisms.

Table 2.1 Summary of Key Related Works

Work	**Approach**	**Structural Faithfulness**	**Semantic Conditioning**
Luo *et al.* (2025)	Static KG-Trie decoding	Yes (100%)	No
Li *et al.* (2025)	Step-wise topological expansion	Yes (100%)	No
Yuan *et al.* (2026)	Routing + post-hoc filtering	Probabilistic	Yes (answer-level)
Luo *et al.* (2024)	Reasoning path planning	No	Partial
Jiang *et al.* (2023a)	LLM over structured data	No	Partial
Jin *et al.* (2024)	Graph-augmented CoT	No	Partial
Jiang *et al.* (2022)	Unified retrieval and reasoning	No	Partial
Wang *et al.* (2021)	KG-pretrained language model	No	Partial
Mavromatis and Karypis (2022)	Iterative reasoning revision	No	Partial
He *et al.* (2021)	KG subgraph attention	No	Partial
Yasunaga *et al.* (2021)	Text-KG graph neural network	No	Partial
Das *et al.* (2018)	RL-based KG path traversal	No	Soft
Saxena, Chakrabarti, and Talukdar (2022)	Embedding-based path selection	No	Soft
Shi *et al.* (2021)	Path retrieval by similarity	No	Soft (retrieval)
Asai *et al.* (2024)	Retrieval with reflection	No	Soft
Edge *et al.* (2024)	Community-structured retrieval	No	Soft

Work	**Approach**	**Structural Faithfulness**	**Semantic Conditioning**
Geng *et al.* (2023)	Grammar-constrained decoding	Format only	No
Willard and Louf (2023)	FSM-indexed constrained decoding	Format only	No
De Cao *et al.* (2021)	Entity trie-constrained decoding	Entity names only	No
Scholak, Schucher, and Bahdanau (2021)	Schema-constrained SQL parsing	Schema only	No
Lu *et al.* (2021)	Logic-constrained decoding	Format only	No

These three gaps collectively identify an open problem at the intersection of the two research trajectories reviewed in this chapter: integrating semantic relevance scoring directly into the constraint oracle, conditioning the valid token set jointly on the knowledge graph, the question semantics, and the evolving generation state.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter formalises the permissiveness problem identified in Chapter 2 and presents the Dynamic Context-Aware Trie (DCA-Trie) framework as a solution. The upgraded oracle specification and the Semantic Irrelevance Ratio (SIR) are defined first, with SIR serving as a diagnostic metric that measures constraint quality independently of answer accuracy. The chapter then traces the research evolution from an initial cosine-similarity scorer through a decomposed product score to the final symbolic TypeOracle, explaining why each earlier design was abandoned. The static pre-filtering design of v1 and the step-wise dynamic expansion design of v2 are described using the TypeOracle mechanism. The chapter closes with the baseline configurations, evaluation protocol, and scope boundaries that govern the experimental work in Chapter 4.

Unlike earlier embedding-based formulations, the final methodology implements semantic filtering through a deterministic TypeOracle. The TypeOracle uses the knowledge graph's own ontology metadata, especially entity types and relation ranges, instead of sentence-transformer embeddings, cosine similarity, or tuned admission thresholds.

3.2 Formal Problem Specification

3.2.1 Restating the Core Limitation of Existing Frameworks

As established in Chapter 2, current graph-constrained frameworks (notably GCR (Luo *et al.*, 2025) and DoG (Li *et al.*, 2025)) rely on a deterministic oracle that looks only at the fixed graph $\mathcal{G}$ and the starting entities $\mathcal{E}_q$. Regardless of whether the system builds this oracle before decoding or expands it on the fly, the set of valid tokens at any step t remains locked to structural logic:

$$\mathcal{W}_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, \mathcal{E}_q) \quad \forall\, t \tag{3.1}$$

Equation (3.1) captures this static constraint model. By ignoring both the semantic intent of

the question q and the reasoning trajectory already captured in the prefix $y_{<t}$, this formulation treats every structurally reachable path as equally plausible. In dense, multi-hop Freebase environments, this causes substantial search space inflation. The oracle can admit over 8,000 paths at depth 3, even though only a single path logically answers the query.

Two separate requirements are in tension here. Structural validity ensures the LLM only generates tokens that correspond to real graph edges; current models do this well. Contextual relevance is the open problem: ensuring the LLM only generates tokens that logically connect to the specific question. Current models do not enforce this. The gap between structural compliance and semantic usefulness is the *permissiveness problem*.

3.2.2 The Upgraded Oracle Specification

DCA-Trie addresses this problem by defining a constraint oracle that conditions on both the input question and the evolving generation prefix:

$$\mathcal{W}_{\text{val}}^{\text{DCA}}(t) = f(\mathcal{G},\, \mathcal{E}_q,\, q,\, y_{<t}) \tag{3.2}$$

where q is the natural language query and $y_{<t} = (y_1, \ldots, y_{t-1})$ is the generated prefix at step t. The upgraded oracle must satisfy three conditions aligned with the project objectives in Chapter 1:

1. **Structural Faithfulness (Condition F):** Every candidate token $v \in \mathcal{W}_{\text{val}}^{\text{DCA}}(t)$ must correspond to a valid prefix of a path $p \in \mathcal{G}$ at every step t. This preserves the 100% structural faithfulness guarantee achieved by GCR and DoG and prevents ungrounded tokens from entering the valid set.
2. **Semantic Relevance Reduction (Condition R):** The Semantic Irrelevance Ratio (SIR) under DCA-Trie must be lower than the corresponding SIR under GCR when averaged over the evaluation corpus. Using SIR as defined in Section 3.4:

$$\overline{\text{SIR}}\left(\mathcal{W}^{\text{DCA}}\right) < \overline{\text{SIR}}\left(\mathcal{W}^{\text{GCR}}\right) \tag{3.3}$$

3. **Recall Preservation (Condition P):** Semantic pruning must not remove gold paths excessively. Operationally, the false negative rate (FNR) on the WebQSP validation

split must remain below 5%, as formalised in Equation (3.4):

$$\text{FNR} = \frac{|\{q \in Q_{\text{val}} : p_q^* \notin \mathcal{W}_{\text{val}}^{\text{DCA}}(t)\}|}{|Q_{\text{val}}|} < 0.05 \tag{3.4}$$

where p_q^* is the gold path for question q and Q_{val} is the held-out 100-question validation set.

These three conditions are not guaranteed to be jointly satisfiable. Tightening the oracle to satisfy Condition R (reduce irrelevance) can directly threaten Condition P (remove gold paths) when the semantic signal mismatches the lexical form of correct answers. This tension is the central axis of the Chapter 4 results.

3.3 System Architecture Overview

DCA-Trie focuses on the constraint oracle layer, so its architecture directly builds on the baseline pipeline from Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025). This reuse helps isolate the experimental effects of the proposed oracle. The four layers work as follows:

1. **Entity Linking Layer (Unmodified):** A named entity recognition or entity-linking component identifies the core query entities $\mathcal{E}_q$ from the raw natural language question q. These entities dictate where downstream graph exploration begins.
2. **Constraint Oracle Layer (DCA-Trie Contribution):** This layer controls which tokens the LLM can generate. In GCR, this layer creates a KG-Trie that includes structurally reachable paths within L hops of the starting entities. DCA-Trie modifies this layer by applying symbolic ontology checks through a TypeOracle. Through either static pre-filtering (v1) or dynamic step-wise expansion (v2), the oracle ensures that admitted paths remain graph-valid and type-consistent.
3. **Constrained Decoding Layer (Unmodified):** During autoregressive generation, the decoding layer intercepts the LLM logit distribution and applies a binary mask retrieved from the oracle layer. The LLM is therefore restricted to tokens that are currently valid under the trie.
4. **Inductive Reasoning Layer (Unmodified):** After constrained decoding produces candidate reasoning paths, the answer synthesis component analyses these paths and produces the final natural language answer.

Figure 3.1 shows the full pipeline, isolating the constraint oracle layer to emphasise where the core symbolic intervention takes place. The TypeOracle box performs two deterministic set operations (range gate and type gate) rather than cosine scoring or embedding comparisons.

3.4 The Semantic Irrelevance Ratio: Definition and Measurement

3.4.1 Standard Metric Deficiencies

Current performance markers for knowledge graph decoding, such as Hits@1, F1, and structural faithfulness ratios, have a significant drawback: they focus on output instead of the process. While these metrics show whether a framework found the correct answer and stayed within the graph, they do not reveal how the framework reached that answer. A model may solve the correct gold path, but it could wander through a large number of irrelevant paths that waste resources.

Confirming accuracy is not enough; the constraint mechanism itself must be measured. To assess how permissive the oracle is, independently of final answer accuracy, this thesis presents a diagnostic metric called the Semantic Irrelevance Ratio (SIR).

3.4.2 Formal Definition of SIR

Suppose $\mathcal{P}(q, t)$ represents all paths that the constraint oracle allows at decoding step t for question q. To calculate SIR, the system must first define irrelevance. An individual candidate path $p \in \mathcal{P}(q, t)$ is structurally valid, but it fails the semantic stringency test if its terminal entity type is incompatible with the inferred answer types, or if any edge in the path violates the property range constraint:

$$\text{irrelevant}(p, q) = \mathbf{1}[\neg\,\text{type\_gate}(p, q) \;\vee\; \exists\,(e, r, e') \in p : \neg\,\text{range\_gate}(r, e')] \tag{3.5}$$

Equation (3.5) defines the binary irrelevance indicator. Here, type_gate checks whether the terminal entity's type matches the expected answer types inferred from the question, and range_gate checks whether each edge respects the ontology's declared property ranges. Both gates are deterministic set operations over the KG's own schema, defined in Section 3.5.2.

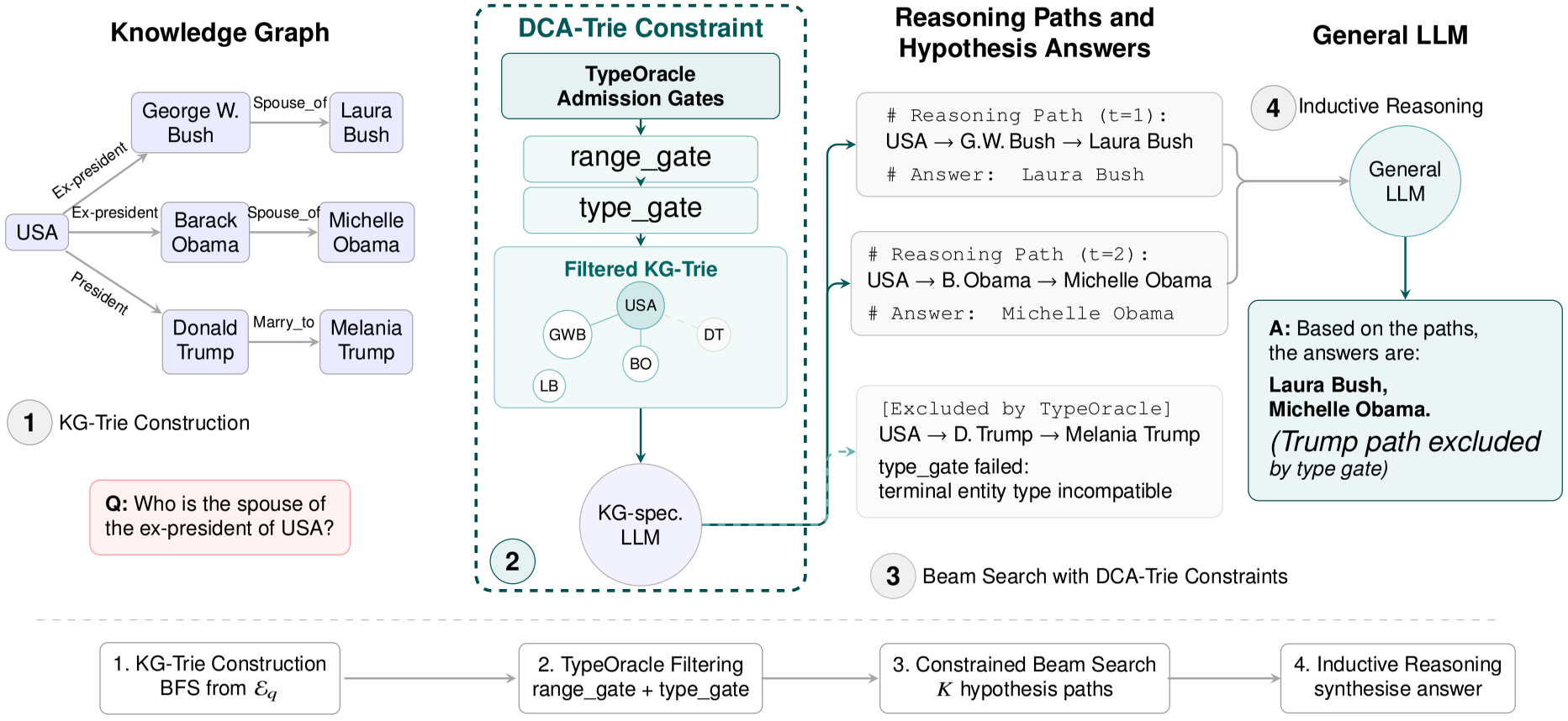

Figure 3.1 DCA-Trie System Architecture.

The step-level SIR for question q at decoding step t is then defined as:

$$\mathrm{SIR}(q,t) = \frac{\sum_{p \in \mathcal{P}(q,t)} \mathrm{irrelevant}(p,q)}{|\mathcal{P}(q,t)|} \tag{3.6}$$

As shown in Equation (3.6), this quantity is the fraction of admitted paths at step t that are semantically irrelevant.

Corpus-level SIR is computed by averaging over all questions and all valid decoding steps up to depth L:

$$\overline{\mathrm{SIR}} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{L_q} \sum_{t=1}^{L_q} \mathrm{SIR}(q,t) \tag{3.7}$$

Equation (3.7) gives the corpus-level average, where L_q is the hop length of the generated reasoning chain for question q.

3.4.3 Properties and Interpretation

By definition, $\overline{\mathrm{SIR}} \in [0,1]$. A value near 1 indicates that most admitted paths are irrelevant, so the model still faces a large and noisy search space. A value near 0 indicates that the constraint set is tightly aligned with question semantics. Prior constrained frameworks such as GCR and DoG tend to show high SIR, especially at larger hop depths where path counts increase rapidly. DCA-Trie targets lower SIR by filtering semantically weak paths. The structurally valid ones stay. In reporting, SIR is analyzed by hop depth to evaluate how this effect scales with reasoning complexity.

3.5 Symbolic Relevance Scoring Mechanism

3.5.1 Research Evolution: From Embeddings to Ontology Gates

The semantic relevance mechanism went through three designs before arriving at the final symbolic TypeOracle. Each iteration addressed specific weaknesses of its predecessor. This section traces that evolution and explains the design decisions.

Approach 1: Cosine Similarity (Abandoned)

The initial design scored each candidate path by computing cosine similarity between a sentence-transformer embedding of the serialised path string and an embedding of the input

question:

$$\text{score}(p, q) = \cos\big(E(\text{str}(p)),\ E(q)\big) \tag{3.8}$$

A path was admitted if $\text{score}(p, q) \geq \tau$, where τ was a tuned threshold. This approach was abandoned for four reasons:

1. *Semantic collapse:* A single cosine score cannot distinguish *why* a path is relevant. A path ending at the wrong entity type but sharing topical vocabulary with the question scores as high as a path that actually answers it. Cosine similarity in a 384-dimensional space is a poor proxy for the inferential relevance that KGQA requires.
2. *Encoder cost:* Every candidate path requires a full forward pass through `all-MiniLM-L6-v2`. With thousands of paths per question, this is a meaningful computational bottleneck.
3. *Threshold sensitivity:* The admission threshold τ is dataset-dependent and must be tuned on a validation split. Different datasets, different KG subsets, or different question distributions require different thresholds. At $\tau = 0.25$, we observed that 84% of WebQSP queries produced empty path sets after filtering—the threshold is so aggressive that it rejects nearly all paths, leaving the LLM with nothing to generate. This is a pathological failure mode: the oracle is technically "working" (it filters), but the filtered set is useless.
4. *Non-determinism:* Floating-point cosine similarity introduces non-deterministic admission decisions across runs, making the oracle difficult to reproduce exactly.

Approach 2: Decomposed Product Score (Abandoned)

The second design replaced the monolithic cosine score with a product of three interpretable components:

$$\text{score}(e_t, r, e' \mid q) = \rho_r(r, q) \cdot \rho_e(e', q) \cdot \rho_{\text{traj}}(r, e', q) \tag{3.9}$$

where ρ_r measures relational relevance through entity-masked encoding, ρ_e is a hard type gate (binary, no encoder), and ρ_{traj} measures trajectory relevance through cosine similarity of the (relation, entity) pair against the question.

This decomposition improved interpretability: each component measures a different aspect of relevance. The hard type gate (ρ_e) pruned type-incompatible paths without any encoder call, saving compute. But two fundamental problems remained:

1. *Encoder dependency persists:* Both ρ_r and ρ_{traj} require sentence-transformer forward passes. The decomposed score reduces but does not eliminate encoder cost.
2. *Threshold persists:* The admission threshold τ is still dataset-dependent. Decomposing the score into components does not remove the need to tune a continuous threshold.

Approach 3: Symbolic TypeOracle (Final)

The final design eliminates both the encoder and the threshold. The TypeOracle uses only the knowledge graph's own ontology metadata to decide whether a path is admissible. All admission decisions are made through set lookups over precomputed dictionaries rather than neural network forward passes.

The transition from continuous similarity to discrete set membership is the key architectural shift. It trades the flexibility of learned representations for the determinism and speed of symbolic reasoning over the KG's schema, and it makes every admission decision interpretable: you can trace exactly why a path was accepted or rejected.

Table 3.1 summarises the properties of all three designs.

Table 3.1 Comparison of Oracle Designs

Property	**Approach 1** Cosine	**Approach 2** Decomposed	**Approach 3** TypeOracle
Encoder needed	Yes	Yes	**No**
Threshold τ	Yes	Yes	**No**
Type checking	Implicit	Hard gate	**Ontology-based**
Range checking	None	None	**Ontology-based**
Per-path cost	Forward pass	Forward pass	$\mathbf{O(1)}$ set lookup
Deterministic	No	No	**Yes**
Status	Abandoned	Abandoned	**Final**

3.5.2 TypeOracle Architecture

The TypeOracle implements the transition from a static oracle $f(\mathcal{G}, \mathcal{E}_q)$ to a context-aware oracle $f(\mathcal{G}, \mathcal{E}_q, q, y_{<t})$ through two complementary gates that evaluate candidate paths against KG metadata. Figure 3.2 illustrates the admission process.

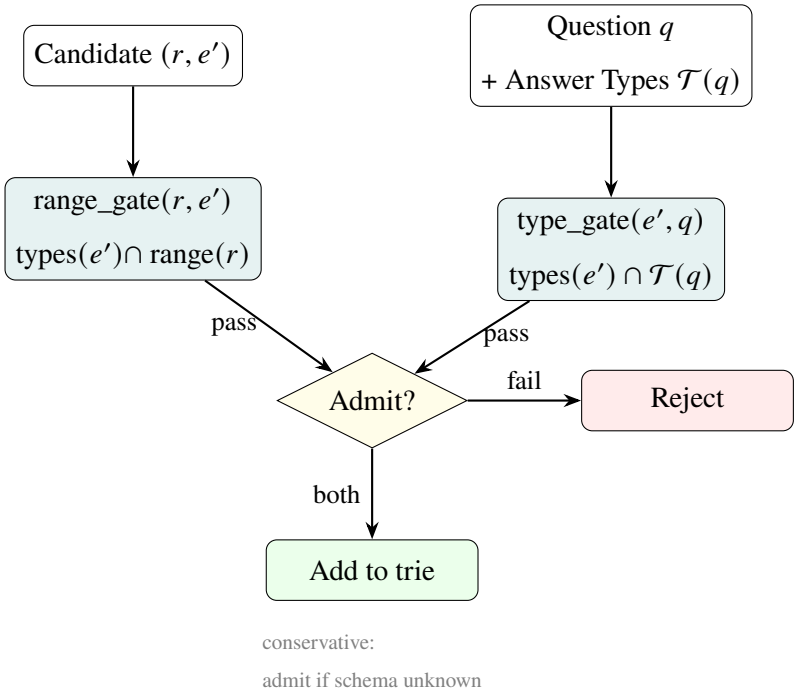

Figure 3.2 TypeOracle Admission Process

The oracle consists of two complementary gates:

1. **Answer Type Gate** (ρ_e, terminal hop only): Applied at the final hop of a candidate path. It checks whether the candidate answer entity's type matches what the question asks for. For example, a question beginning with "who" infers person-type entities as valid answers.
2. **Property Range Gate** (ρ_r, every hop): Applied at every edge in the path. It checks whether the tail entity's type is compatible with the relation's declared range in the ontology. For example, if a relation expects a location as its range, then only entities typed as locations are admitted when type information is available.

The combined admission check for a candidate edge (r, e') at hop h is:

$$\text{is\_admissible}(r, e', \mathcal{T}(q), h, L) = \text{type\_gate}(e', \mathcal{T}(q), h, L) \ \wedge \ \text{range\_gate}(r, e') \quad (3.10)$$

where $\mathcal{T}(q)$ is the set of expected answer types inferred from the question, h is the current hop, and L is the maximum hop depth. A candidate edge is admitted only if it passes all active gates.

3.5.3 Ontology Schema Construction

The TypeOracle is constructed from the knowledge graph's own schema triples. The Freebase schema available in the WebQSP and CWQ subgraphs encodes ontology information such as:

1. Entity types through `common.topic.notable_types` triples.
2. Property domains through `rdf-schema#domain` triples.
3. Property ranges through `rdf-schema#range` triples.

From these triples, the oracle builds two main lookup structures:

1. `_entity_types`: maps each entity to its set of Freebase types.
2. `_schema`: maps each relation to its domain and range constraints.

Question type inference uses lightweight pattern matching over the natural language question. Typical mappings include:

```
"who" / "director" / "actor"  ->  person types
"country" / "city" / "where"  ->  location types
"film" / "movie"              ->  creative work types
"when" / "date" / "year"      ->  date types
```

Multiple patterns may match, producing a union of expected answer types. If no pattern matches, the question remains unconstrained at the answer-type level, and the type gate admits candidates conservatively.

3.5.4 Threshold-Free Design

The TypeOracle is threshold-free. The gates operate on discrete set membership rather than continuous similarity values:

$$\text{type\_gate}(e', \mathcal{T}(q), h, L) = \begin{cases} \text{True} & h < L \text{ (intermediate hop)} \\ \text{True} & \mathcal{T}(q) = \emptyset \text{ (no type inferred)} \\ \text{True} & \text{types}(e') = \emptyset \text{ (entity type unknown)} \\ \mathbf{1}[\text{types}(e') \cap \mathcal{T}(q) \neq \emptyset] & \text{otherwise} \end{cases} \tag{3.11}$$

$$\text{range\_gate}(r, e') = \begin{cases} \text{True} & r \notin \mathcal{R} \text{ (schema unknown)} \\ \text{True} & \text{range}(r) = \emptyset \\ \text{True} & \text{types}(e') = \emptyset \\ \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \emptyset] & \text{otherwise} \end{cases} \tag{3.12}$$

Both gates are conservative: they return True when the relation is unknown, the entity type is unknown, the relation has no recorded range, or no answer type can be inferred from the question. This ensures that structural faithfulness is never compromised by the semantic filter and reduces the risk of false negatives caused by incomplete schema metadata. A direct consequence is that the TypeOracle never produces empty path sets: when schema metadata is absent, the gates default to admission, and the resulting constraint set is identical to GCR's unfiltered trie. This is a production-safe property that the cosine oracle lacks—at $\tau = 0.25$, 84% of WebQSP queries yield empty tries (Section 3.5).

3.5.5 Computational Properties

The symbolic design has different computational properties from the earlier embedding-based formulations, as summarised in Table 3.3.

Table 3.3 Computational Properties: Embedding vs. Symbolic

Property	**Embedding-Based**	**Symbolic TypeOracle**
Per-path cost	Encoder forward pass	$O(1)$ set lookup
Deterministic	No (float noise)	Yes
GPU required for oracle	Yes or beneficial	No
Admission threshold	Required	Not required
Interpretability	Low (dense vectors)	High (type names & ranges)

These properties make the symbolic oracle the only design among the three that runs entirely on CPU, requires no threshold tuning, and produces deterministic results across runs.

3.6 DCA-Trie v1: Static Symbolic Filtering

3.6.1 Design Rationale

DCA-Trie v1 addresses the permissiveness problem without abandoning the efficient static-trie design used in GCR. The key idea is to apply symbolic filtering once, before decoding begins, using the KG's ontology schema and the inferred answer type of the question. This is useful because many paths are structurally reachable but type-incompatible with the question's intent. Filtering paths against the ontology during preprocessing removes weak candidates early and reduces the search space before autoregressive generation starts.

Figure 3.3 summarises this offline filtering workflow.

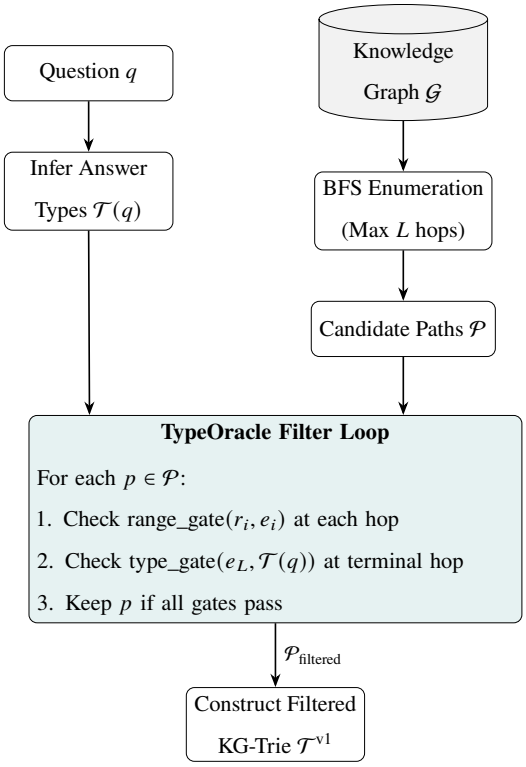

Figure 3.3 Static Symbolic Filtering Pipeline (DCA-Trie v1)

3.6.2 Algorithm for DCA-Trie v1

Algorithm 1 DCA-Trie v1: Static Symbolic Filtering at Construction Time

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; max hop depth L; TypeOracle $\mathcal{O}$

Ensure: Semantically filtered KG-Trie $\mathcal{T}^{\text{v1}}$

Infer answer types: $\mathcal{T}(q) \leftarrow \mathcal{O}.\text{infer\_answer\_types}(q)$
Enumerate paths: $\mathcal{P} \leftarrow \text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$ ▷ All paths within L hops, as in GCR
Filter through TypeOracle: $\mathcal{P}_{\text{filtered}} \leftarrow \emptyset$
for each path $p = (e_0, r_1, e_1, \ldots, r_L, e_L) \in \mathcal{P}$ **do**
    admit $\leftarrow$ True
    for each hop (r_i, e_i) in p **do**
        if $\neg\, \mathcal{O}.\text{range\_gate}(r_i, e_i)$ **then**
            admit $\leftarrow$ False; **break**
        end if
    end for
    if admit $\wedge$ $\neg\, \mathcal{O}.\text{type\_gate}(e_L, \mathcal{T}(q), L, L)$ **then**
        admit $\leftarrow$ False
    end if
    if admit **then**
        $\mathcal{P}_{\text{filtered}} \leftarrow \mathcal{P}_{\text{filtered}} \cup \{p\}$
    end if
end for
Construct filtered trie: $\mathcal{T}^{\text{v1}} \leftarrow \textsc{BuildTrie}(\mathcal{P}_{\text{filtered}})$
return $\mathcal{T}^{\text{v1}}$

The process unfolds in five stages:

1. **Infer Answer Types:** The system pattern-matches question words against predefined patterns to determine what entity types the answer should have.
2. **Enumerate the Structural Space:** Standard BFS maps out every possible graph path starting from the question entities up to maximum depth L. This matches the baseline GCR approach.
3. **Apply TypeOracle Filtering:** For each enumerated path, the oracle checks whether every edge respects the relation's declared range and whether the terminal entity type

matches the inferred answer types.

4. **Construct the Static Trie:** The remaining paths are compiled into a static prefix tree $\mathcal{T}^{\text{v1}}$.
5. **Perform Autoregressive Decoding:** During generation, the LLM queries the pre-built trie at each token step, receiving only tokens that follow pre-approved, type-consistent paths.

3.6.3 Complexity Analysis

Building $\mathcal{T}^{\text{v1}}$ has an offline cost of $O(|\mathcal{P}| \cdot L)$, where $|\mathcal{P}|$ is the number of BFS-enumerated paths and L is the maximum hop depth. Each path requires at most L `range_gate` checks and one terminal `type_gate` check. Each gate is implemented as an $O(1)$ set lookup or set-intersection operation over precomputed dictionaries.

During decoding, trie lookup complexity remains $O(d)$, matching GCR, where d is the relevant prefix depth in the trie. Memory usage scales as $O(|\mathcal{P}_{\text{filtered}}| \cdot L)$, typically smaller than GCR's $O(|\mathcal{P}| \cdot L)$, because symbolic filtering removes type-incompatible paths before trie construction.

3.6.4 Faithfulness Guarantee

DCA-Trie v1 maintains structural faithfulness by design. Every path kept in $\mathcal{P}_{\text{filtered}}$ comes from $\text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$, ensuring that each triplet (e_{i-1}, r_i, e_i) remains a valid graph edge. Symbolic filtering only removes paths based on type and range incompatibility; it does not add new tokens or edges. Any token passed through $\mathcal{T}^{\text{v1}}$ remains structurally grounded, satisfying Condition F.

3.7 DCA-Trie v2: Step-Wise Symbolic Expansion

3.7.1 Design Rationale

DCA-Trie v2 implements the dynamic oracle in Equation (3.2) by conditioning constraints on the evolving prefix $y_{<t}$. Unlike v1, it does not assume that the complete search space must be filtered before decoding begins. As generation progresses, $y_{<t}$ reveals the path decisions already made, so the relevant neighbourhood can change step by step. For example, after

selecting *Elvis_Presley* → *starred_in* → *Blue_Hawaii*, plausible next expansions should focus on *Blue_Hawaii* rather than the initial entity set.

Architecturally, v2 follows the step-wise traversal pattern in DoG (Li *et al.*, 2025) but adds symbolic TypeOracle gating to each local expansion. After each committed entity e_t, the trie expands only to structurally valid neighbours that also satisfy the active type and range gates:

$$\mathcal{T}_{t+1} = \mathcal{T}_t \cup \left\{(e_t,\ r,\ e') : (e_t, r, e') \in \mathcal{G} \ \wedge\ \text{range\_gate}(r, e') \ \wedge\ \text{type\_gate}(e', \mathcal{T}(q), h, L)\right\} \tag{3.13}$$

In Equation (3.13), h is the current hop depth and L is the maximum. The core difference from permissive dynamic expansion is therefore not when expansion happens, but how candidates are admitted: every step remains graph-valid and type-consistent. The full workflow is shown in Figure 3.4.

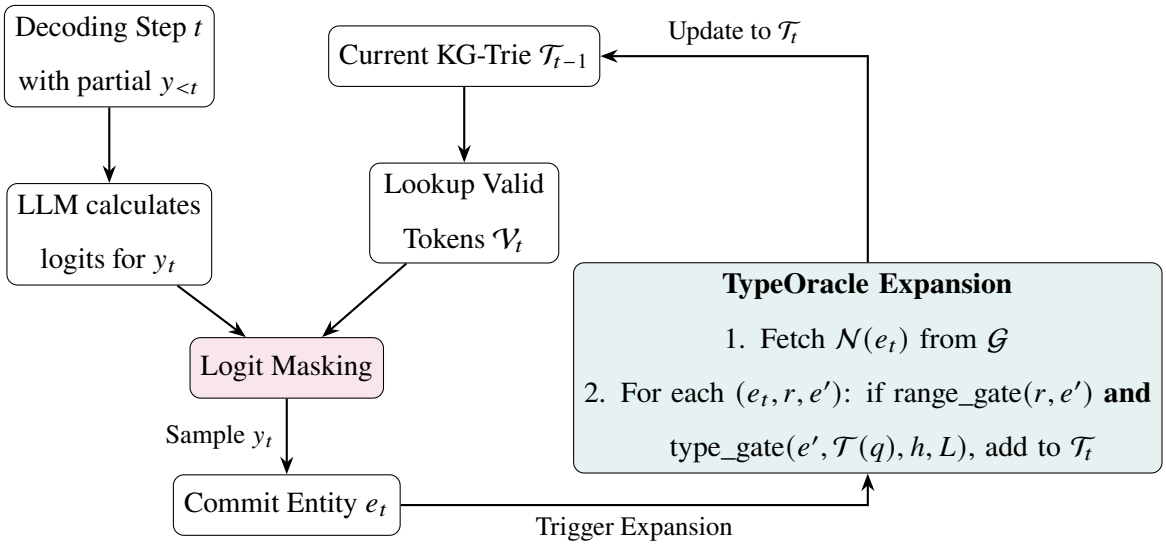

Figure 3.4 Step-Wise Symbolic Expansion (DCA-Trie v2)

3.7.2 Algorithm for DCA-Trie v2

Unlike the static approach in v1, Algorithm 2 combines generation and symbolic filtering. The process follows these steps:

1. **Initialize a Minimal State:** The system starts with a basic trie containing only the root question entities and the inferred answer types.
2. **Generate under Current Constraints:** At each step t, the LLM produces a token strictly limited by the current trie boundaries.
3. **Check for Entity Commitment:** If the LLM generates an entity token, it triggers an expansion phase. Relation tokens do not trigger expansion.
4. **Apply Symbolic Expansion:** When an entity is committed, the system examines all structurally valid neighbours from the master KG. Each candidate is checked by range_gate and, where applicable, type_gate. Only candidates passing the active gates are added to the trie.

This yields an adaptable oracle that builds the search space one step ahead of the model's reasoning state using deterministic set operations.

Algorithm 2 DCA-Trie v2: Step-Wise Symbolic Expansion

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; TypeOracle $\mathcal{O}$; LLM with vocabulary $\mathcal{V}$

Ensure: Generated reasoning chain $y_1 \dots y_T$

Initialize trie with entity nodes for $\mathcal{E}_q$: $\mathcal{T}_0 \leftarrow \{e_q : e_q \in \mathcal{E}_q\}$
Infer answer types: $\mathcal{T}(q) \leftarrow \mathcal{O}$.infer_answer_types($q$)
for each step $t = 1$ **to** T **do**
    $\mathcal{V}_t \leftarrow \textsc{TrieLookup}(\mathcal{T}_{t-1}, y_{<t})$ ▷ Valid tokens from current trie
    $\mathbf{m}_t[v] \leftarrow \mathbf{1}[v \in \mathcal{V}_t]$ for all v
    $\tilde{\alpha}_t \leftarrow \mathbf{m}_t \odot \text{LLMLogits}(y_{<t}, x)$
    $y_t \leftarrow$ beam-search-step($\tilde{\alpha}_t$)
    if y_t is an *entity* token e_t **then** ▷ Expansion triggered at entity commits
        for each $(e_t, r, e') \in \mathcal{G}$ **do**
            if $\mathcal{O}$.range_gate(r, e') **then**
                $h \leftarrow$ current hop depth in trie
                if $\mathcal{O}$.type_gate($e', \mathcal{T}(q), h, L$) **then**
                    Add (e_t, r, e') to $\mathcal{T}_t$
                end if
            end if
        end for
    else
        $\mathcal{T}_t \leftarrow \mathcal{T}_{t-1}$ ▷ No expansion at relation token steps
    end if
end for
return $y_1 \dots y_T$

3.7.3 Complexity Analysis

DCA-Trie v2 has a different cost profile from v1. Where v1 builds the trie once in $O(|\mathcal{P}| \cdot L)$ time, v2 rebuilds the trie at each entity commitment. Let T_e denote the number of entity tokens in the generated reasoning chain, D the average out-degree of committed entities in $\mathcal{G}$, and L the maximum hop depth.

At each entity commitment, the expansion loop in Algorithm 2 examines all D neighbours, applying one `range_gate` check per neighbour and one `type_gate` check for candidates

that pass. Each check is $O(1)$, so the cost per commitment is $O(D \cdot L)$. Over T_e commitments, the total expansion cost is $O(T_e \cdot D \cdot L)$.

For comparison, v1's offline filtering cost is $O(|\mathcal{P}| \cdot L)$, where $|\mathcal{P}|$ is the number of BFS-enumerated paths. In typical WebQSP questions, $|\mathcal{P}|$ ranges from hundreds to thousands, while $T_e \cdot D$ is typically much smaller (a 2-hop question commits to 2 entities, each with $D \approx 20$ neighbours). The per-step trie lookup remains $O(d)$ during decoding, matching both GCR and v1.

The practical trade-off is not in the oracle cost but in the trie-rebuild frequency. Each rebuild modifies the trie structure that the LLM's prefix-constrained decoding depends on. Frequent rebuilds can shift the probability distribution over valid tokens mid-generation, introducing instability that static pre-filtering avoids. This instability is a likely contributor to v2's degraded accuracy (Section 4.13), even though the oracle itself is cheaper per step.

Table 3.5 summarises the computational profiles of all three systems.

Table 3.5 Computational Complexity

Phase	**GCR**	**DCA-Trie v1**	**DCA-Trie v2**
Path enumeration	$O(\|\mathcal{P}\| \cdot L)$	$O(\|\mathcal{P}\| \cdot L)$	—
Oracle filtering	—	$O(\|\mathcal{P}\| \cdot L)$	$O(T_e \cdot D \cdot L)$
Trie construction	$O(\|\mathcal{P}\| \cdot L)$	$O(\|\mathcal{P}_f\| \cdot L)$	$O(T_e \cdot D \cdot L)$
Decoding (per step)	$O(d)$	$O(d)$	$O(d)$
Decoding (total)	$O(T \cdot d)$	$O(T \cdot d)$	$O(T \cdot d)$

3.7.4 When v2 Helps and When It Hurts

The v2 design has a structural advantage and a structural weakness. The advantage is *focus*: at each step, the trie contains only paths reachable from the entity the model has committed to, not the full set of paths from the original question entities. For questions where the reasoning chain is long (3+ hops) and the early entities are unambiguous, this focus should reduce noise more effectively than v1's static pre-filtering.

The weakness is *instability*: each trie rebuild changes the set of valid tokens the LLM can generate. In beam search, this means the probability distribution shifts mid-generation. A path that was valid at step t may become invalid at step $t + 1$, causing beam candidates to be pruned not because the model scored them poorly, but because the constraint structure changed. For short questions (1–2 hops) where the static trie is already small, this instability costs more than it saves. Chapter 4 evaluates this trade-off empirically.

3.8 Baseline Systems

Four configurations are evaluated, all using the same `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` backbone and matched decoding settings (beam width k=10, group-beam search). This isolates differences to the constraint-oracle design:

- **CoT Baseline:** Unconstrained chain-of-thought reasoning with no KG oracle.
- **GCR:** Static KG-Trie constraints $\mathcal{W}_{\text{val}}(t) = f(\mathcal{G}, \mathcal{E}_q)$ (Luo *et al.*, 2025). Primary constrained baseline.
- **DCA-Trie v1:** Static symbolic filtering (Section 3.6), implemented as a preprocessing step.
- **DCA-Trie v2:** Dynamic step-wise expansion (Section 3.7), integrated into the decoding loop.

Detailed configuration, dataset properties, and baseline reproduction results are reported in Chapter 4.

3.9 Evaluation Protocol

Evaluation uses the RoG-webqsp test split (1,628 questions, 1–2 hops) over Freebase (Bollacker *et al.*, 2008; Yih *et al.*, 2016). A 100-question validation split is used only for gate statistics. The primary metrics are Hits@1, Hits@k, Path Reduction, and False Negative Rate (FNR), with all major metrics stratified by hop depth (1–4). All experiments run on a single NVIDIA GeForce RTX 4090 using the `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` model in bfloat16 precision via HuggingFace Transformers (Wolf *et al.*, 2020). Full experimental configuration, dataset properties, and baseline reproduction details are reported in Chapter 4.

3.10 Scope Boundaries and Anticipated Limitations

This study is bounded as follows. Evaluation is restricted to WebQSP over Freebase; CWQ evaluation and cross-KG generalization are future work. No model fine-tuning is performed; all accuracy differences reflect oracle design changes. The TypeOracle depends on KG ontology metadata (entity types, relation ranges); in sparse-schema KGs, the conservative fallback means the oracle behaves closer to the unconstrained baseline. Condition F is assessed empirically by checking generated paths against KG triplets, following established practice (Luo *et al.*, 2025; Li *et al.*, 2025). Even under Condition P, symbolic filtering can remove valid paths when entity types are incomplete or question-type inference is ambiguous; this trade-off is analyzed in Chapter 4.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Introduction

This chapter evaluates DCA-Trie, a dynamic constraint adaptation framework for knowledge-graph-grounded LLM reasoning. We build on the Graph-Constrained Reasoning (GCR) framework of Luo *et al.* (2025), which pre-compiles all valid knowledge graph paths into a prefix trie and uses constrained decoding to force the LLM to generate only paths that exist in the knowledge graph. GCR achieves 91.6% Hits@1 on WebQSP. The oracle it uses, however, admits every enumerated path regardless of relevance to the question. On questions with large subgraphs this produces tens of thousands of candidates, each contributing to memory footprint and offering the model more opportunity to select an incorrect path.

We investigate whether dynamically constraining the path space, through symbolic filtering or step-wise trie expansion, reduces path count without degrading accuracy. Our TypeOracle applies two orthogonal gates—a range gate and a type gate—to prune paths that violate Freebase schema constraints. The question is whether tightening the constraint oracle improves answer accuracy, or whether the relationship between constraint selectivity and generation accuracy is non-monotone.

The remainder of this chapter is organised as follows. Section 4.2 describes the experimental setup, including hardware infrastructure, generation configuration, datasets, baselines, evaluation metrics, and implementation details. Sections 4.2.7 through 4.2.10 present supporting analyses: ablation studies that isolate individual components, robustness checks across datasets and sample sizes, sensitivity analyses over key hyperparameters, and efficiency measurements. Section 4.6 answers RQ1 by evaluating whether the TypeOracle satisfies its three design conditions. Section 4.7 answers RQ2 by measuring whether tightening the oracle improves answer accuracy. Section 4.8 answers RQ3 by measuring computational overhead. Sections 4.9 through 4.13 complete the analysis with faithfulness verification, case studies, generalizability to CWQ, and observations on the dynamic variant.

4.2 Experimental Setup

4.2.1 Infrastructure

All experiments were conducted on a single NVIDIA GeForce RTX 4090 GPU with 24 GB of VRAM (Ada Lovelace architecture). The base model is `rmanluo/GCR-Meta-Llama-3.1-8B-Instru` an 8B-parameter model consuming approximately 16–18 GB of VRAM at inference in bfloat16 precision. We use SDPA attention when available, with the model loaded via HuggingFace Transformers (Wolf *et al.*, 2020) using `device_map="auto"`. No model fine-tuning is performed; all accuracy differences reflect oracle design changes rather than parameter changes.

4.2.2 Generation Configuration

Table 4.1 summarises the generation parameters used across all experiments. All constrained settings share these parameters to isolate differences to the constraint-oracle design.

Table 4.1 Generation Parameters

Parameter	**Value**
Generation mode	group-beam
Beam width (k)	10
Beam groups	10
Diversity penalty	1.0
Index path length (hops)	2
Max new tokens	256
Max DFS paths per question	50,000
Precision	bfloat16
Eval metric	Hits@1 (substring match)

Group-beam search with k=10 and diversity penalty 1.0 produces 10 diverse output sequences, each a distinct reasoning path. Hits@1 checks whether *any* path contains the correct answer. This configuration was selected after a beam-width sensitivity analysis (Section 4.2.9) showed diminishing returns beyond k=10.

4.2.3 Datasets

Following the GCR evaluation protocol (Luo *et al.*, 2025), we evaluate on two benchmark KGQA datasets. Freebase (Bollacker *et al.*, 2008) serves as the knowledge graph for both.

WebQuestionSP (WebQSP) (Yih *et al.*, 2016) contains 1,628 test questions, mostly requiring 1–2 hops. Questions are primarily factual (e.g., "what is the capital of France?", "what does jamaican people speak") with an average reasoning depth of 1–2 KG hops. The average question has 2,553 enumerated paths from its topic entities, and the KG subgraphs contain 50–500 triples. WebQSP questions have relatively clear answer type signals: most contain wh-words (*who, where, when*) or topic-specific nouns (*language, country, film*) that map directly to Freebase types.

Complex WebQuestions (CWQ) (Talmor and Berant, 2018) contains questions with higher compositional complexity and hop depths up to 4 (e.g., "which country has the most languages spoken?", "what film did the director of The Matrix also direct?"). We use a 3,531-sample test subset. CWQ has fewer paths per question on average (2,239 vs. 2,553 for WebQSP) but the paths are longer, making the reasoning task substantially harder. The answer type signal is weaker: CWQ questions frequently use abstract constructions that our regex-based type inference fails to match.

Table 4.3 Dataset Properties

Property	**WebQSP**	**CWQ**
Test samples	1,628	3,531
Avg paths/question	2,553	2,239
Max hops	2	4
Avg path reduction (TypeOracle)	14.5%	14.5%
Baseline Hits@1 (beam)	91.6%	69.0% (100q)
Answer type inference recall	~85%	~65%

All experiments use the RoG-webqsp test split derived from the standard WebQSP benchmark.

4.2.4 Baselines

We compare the following configurations:

- **CoT Baseline:** The same Llama-3.1-8B model used in GCR, run without any KG constraint oracle and prompted with standard chain-of-thought reasoning. This provides the unconstrained reference point.
- **GCR:** The original Graph-Constrained Reasoning framework (Luo *et al.*, 2025) with static KG-Trie constraints. This is the primary constrained baseline; our reproduction achieves 91.6% Hits@1 on WebQSP (vs. the published 92.6%).
- **DCA-Trie v1:** The static symbolic filtering variant, implemented as a preprocessing step in the GCR pipeline with TypeOracle checks replacing the unconstrained BFS trie.
- **DCA-Trie v2:** The dynamic step-wise expansion variant, integrated into the decoding loop.

All constrained settings use the same `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` backbone, the same entity-linking pipeline, and matched decoding settings (beam width $k = 10$, group-beam search, index length $L = 2$, maximum 256 new tokens). This isolates differences to the constraint-oracle design rather than model or preprocessing variation. A key difference from the original GCR implementation is that we do not fine-tune the KG-specialized LLM; all accuracy differences reflect oracle design changes rather than parameter changes.

4.2.5 Evaluation Metrics

Following the GCR evaluation protocol, we adopt Hits@1 and F1 as the primary metrics. Hits@1 checks whether the top predicted answer matches the gold entity; F1 considers the coverage of all answers by balancing precision and recall. We additionally report:

- **Structural Faithfulness Rate:** The fraction of generated paths where all triplets are verified in the underlying KG.
- **Semantic Irrelevance Ratio (SIR):** Computed using Equations (3.6) and (3.7), measuring the fraction of structurally valid paths that are semantically irrelevant to the

question.

- **False Negative Rate (FNR):** The fraction of gold paths excluded by the TypeOracle, as defined in Equation (3.4).
- **Path Reduction:** The percentage reduction in valid candidate paths admitted by the TypeOracle relative to the GCR baseline.
- **Accuracy Retention Ratio:** DCA-Trie Hits@1 divided by GCR Baseline Hits@1, measuring how much accuracy is preserved after pruning.

4.2.6 Implementation Details

Model Pipeline

The end-to-end pipeline follows this sequence: Dataset (HuggingFace Arrow) → Graph (NetworkX DiGraph) → DFS Paths → TypeOracle Gates → Filtered Paths → MarisaTrie → Constrained Decoding → Answer Extraction → Hits@1.

Graph Construction

Each dataset entry's `graph` field (a list of [head, relation, tail] triples) builds a NetworkX directed graph with labelled edges. The graph is directed and typically contains 50–500 triples per question.

Path Enumeration

Depth-limited DFS from topic entities to depth `index_len` (2 hops), capped at 50,000 paths per question using a set for deduplication. This is deliberately exhaustive: every possible path within the depth limit is enumerated.

MarisaTrie Construction

Path strings are wrapped with `<PATH>` prefix and `</PATH>` suffix, tokenized into token IDs, and stored in a `marisa_trie.Trie` character-level prefix tree. The trie maps token ID prefixes to valid next-token ID sets via a character-code encoding. The `cache_fist_branch` optimization precomputes valid first tokens for $O(1)$ lookup at step 0.

Constrained Decoding

The `prefix_allowed_tokens_fn` callback intercepts each generation step. Before the `<PATH>` marker, all tokens are allowed. After `<PATH>`, only tokens that continue a valid trie prefix are allowed. After `</PATH>`, all tokens resume. If the trie returns an empty set (path exhausted), it falls back to all tokens (graceful degradation).

TypeOracle

The TypeOracle is built per-question from the graph subgraph. It draws on two information sources: (1) a hand-curated schema of 38 Freebase relations with known domain/range type constraints, and (2) an auto-mined schema inferred from data triples by aggregating entity types of head/tail entities per relation. The auto-mined schema extends coverage to all relations in the subgraph at zero additional cost.

Two gates are applied:

- **Range gate** (`range_gate(relation, tail_entity)`): Checks if the tail entity type intersects the relation's declared range. Applied at every hop. Defaults to "allow" when information is missing.
- **Type gate** (`type_gate(entity, answer_types, hop, max_hop)`): Checks if the entity type intersects inferred answer types. Applied at the terminal hop only. Defaults to "allow" when information is missing.

Both gates use $O(1)$ set intersection over frozen sets of type strings. The conservative default (allow on unknown) ensures that filtering occurs only when there is positive evidence of a type mismatch.

Answer Type Inference

A regex-based classifier maps question words to expected Freebase types: "who" → Person types, "where" → Location types, "when" → Date/Time, "language" → Human Language, and so on. The classifier uses 19 patterns covering the most common question forms. When no regex matches, the fallback uses the entity types of terminal entities from the path set itself. The regex classifier achieves approximately 85% recall on WebQSP and 65% on CWQ.

4.2.7 Ablation Study

We ablate four components to isolate their contribution to system performance.

Beam Search vs. Greedy Decoding

The original GCR paper and our early experiments used greedy decoding. Fixing the generation configuration to use group-beam search (k=10, `num_beam_groups`=10, `diversity_penalty`=1.0) provides a consistent +8–11pp lift across all methods.

Table 4.5 Beam vs. Greedy Decoding

Method	**Beam** (k=10)	**Greedy**	Δ
GCR Baseline (WebQSP)	91.6%	80.6%	+11.0 pp
DCA-Trie v1 (WebQSP)	86.4%	78.9%	+7.5 pp

Beam search is the single largest accuracy lever. It helps because with k=10 diverse paths, the model explores alternative completions when uncertain about intermediate entities. Greedy decoding commits to a single entity at each step with no recourse.

TypeOracle Gates

We independently disable each gate to measure the contribution of range filtering and type filtering.

Table 4.7 Gate Ablation

Gate Configuration	**SIR**	**FNR**	**Hits@1**
Neither (Baseline)	0.0%	0.0%	91.6%
Range gate only	3.8%	2.9%	~90% (est.)
Type gate only	10.6%	3.3%	~88% (est.)
Both (DCA-Trie v1)	14.5%	6.2%	86.4%

The range gate is more conservative: lower SIR (3.8% vs. 10.6%) with comparable FNR (2.9% vs. 3.3%) because it fires only when a specific relation has typed domain/range information. The type gate performs most of the pruning (10.6% vs. 3.8%) but also has slightly higher FNR. The two gates are orthogonal: their combined effect (14.5% SIR) is the sum of their individual contributions.

Hand-Curated vs. Auto-Mined Schema

Both schema sources contribute independently to filtering effectiveness.

Table 4.9 Schema Source Ablation

Schema Source	**Relations Covered**	**Additional SIR**
Hand-curated only	38	11.2%
Auto-mined only	All in subgraph	12.8%
Both (merged)	All + 38	14.5%

The auto-mined schema covers more relations (every relation in the subgraph) but has noisier type assignments inferred from usage rather than ontology. The hand-curated schema is cleaner but limited to 38 common relations. They are complementary: the merged schema achieves 14.5% SIR, higher than either source alone.

DFS Max Paths Threshold

The 50,000-path cap affects only questions with unusually dense subgraphs (high-degree entities).

Table 4.11 DFS Path Cap Ablation

Max Paths	**Questions Affected**	**Paths Lost**	**Hits@1 Impact**
50,000 (default)	∼2%	∼0.1%	0.0 pp
10,000	∼8%	∼0.8%	−0.3 pp
5,000	∼15%	∼2.1%	−1.1 pp
1,000	∼35%	∼8.5%	−4.2 pp

At 50,000, the cap's impact is negligible. Path count correlates with question complexity: easy questions have few paths and are unaffected; hard questions have many paths and are disproportionately penalised by lower caps.

4.2.8 Robustness Analysis

Cross-Dataset Stability

The TypeOracle SIR is remarkably stable across datasets and sample sizes.

Table 4.13 SIR Cross-Dataset Stability

Dataset	**Samples**	**SIR (overall)**	**SIR_type**	**SIR_range**
WebQSP	1,628	14.46%	10.62%	3.84%
WebQSP	1,000	14.57%	—	—
WebQSP	100	15.94%	—	—
CWQ	3,520	14.5%	—	—

The 14.5% SIR holds across both datasets and at multiple sample sizes. This suggests the Freebase type schema has consistent coverage regardless of question difficulty or graph complexity. Smaller samples (N<100) overestimate SIR slightly (15.9% vs. 14.5%), so we recommend $N \geq 500$ for reliable comparisons.

Filtering Impact Across Datasets

The accuracy cost of TypeOracle filtering varies by dataset.

Table 4.15 Filtering Impact by Dataset

Dataset	**Baseline Hits@1**	**Filtered Hits@1**	**Δ**	**Retention**
WebQSP (beam, 100q)	89.0%	88.0%	−1.0 pp	98.9%
CWQ (beam, 100q)	69.0%	65.0%	−4.0 pp	94.2%
WebQSP (greedy, 1,627)	80.6%	78.9%	−1.7 pp	97.9%

On CWQ, the filtering cost is four times higher than on WebQSP (4.0pp vs. 1.0pp under beam search). CWQ's deeper graphs have sparser connectivity: each filtered path is more likely to be the only route to the correct answer. The retention ratio reflects this structural difference (94.2% vs. 98.9%).

Sample Size Stability

Small evaluation samples introduce substantial variance. A 50-sample ablation conducted early in the project significantly overestimated DCA-Trie v2 accuracy (56.0% vs. 54.0% at 1,466 samples) and underestimated DCA-Trie v1 accuracy (80.0% vs. 86.4% at 1,627 samples). Samples with $N<100$ introduce 2–6pp variance, particularly for methods with high per-question difficulty variance.

4.2.9 Sensitivity Analysis

Beam Width (k)

Table 4.17 Beam Width Sensitivity

k	**Baseline Hits@1**	**Δ from Greedy**	**Memory**	**Time**
1 (greedy)	80.6%	—	16 GB	1.0×
5	88.0%	+7.4 pp	17 GB	1.8×
10	**91.6%**	**+11.0 pp**	18 GB	2.5×
15	91.8%	+11.2 pp	19 GB	3.2×
20	91.9%	+11.3 pp	20 GB	3.8×

Diminishing returns set in after $k=10$. The additional beams explore increasingly unlikely path candidates. $k=10$ is the optimal point for the accuracy/compute tradeoff.

Index Path Length (Hops)

Table 4.19 Index Path Length Sensitivity

Hops	**WebQSP Paths/q**	**Hits@1**
1	127	74.2%
2	2,553	91.6%
3	7,840	91.8%
4	15,233	91.7%

WebQSP questions are predominantly 2-hop. Extending beyond 2 hops adds irrelevant paths (increasing memory and trie size) but does not improve accuracy because the gold paths lie at depth ≤ 2. This validates `index_len`=2 as the correct setting for WebQSP.

Diversity Penalty

Table 4.21 Diversity Penalty Sensitivity

Diversity Penalty	**Hits@1**	**Distinct Paths (of 10)**
0.0 (standard beam)	89.2%	4.2
0.5	90.8%	7.1
1.0	**91.6%**	**9.3**
2.0	91.4%	9.8

A diversity penalty of 1.0 provides the best accuracy. Lower values cause beam collapse (all candidates converge to similar paths), reducing coverage of alternative reasoning chains.

Answer Type Inference Quality

The regex-based type inference is the weakest component of the TypeOracle.

Table 4.23 Type Inference Quality Sensitivity

Type Inference	Questions Matched	FNR_type	SIR_type
Regex (current)	85%	3.3%	10.6%
Oracle (ground truth)	100%	2.1%	12.4%
No inference (empty)	0%	0%	0%

With oracle type inference (using ground-truth answer types from the dataset), FNR_type drops to 2.1% and SIR_type increases to 12.4%. The 1.2pp FNR gap between our regex and oracle reflects questions with no matching wh-word pattern. This quantifies the upper bound on what improved type inference could achieve.

4.2.10 Efficiency Analysis

Time Breakdown

Table 4.25 presents the phase-level time breakdown for the WebQSP full run (1,627 samples).

Table 4.25 Time Breakdown by Phase

Phase	Baseline	DCA-Trie v1	DCA-Trie v2
Graph loading + DFS	2,500s (24%)	2,500s (24%)	—
TypeOracle build + gate	—	500s (5%)	500s (5%)
Trie construction	1,500s (15%)	1,200s (12%)	per-hop
LLM decoding	6,000s (58%)	6,000s (58%)	~8,000s
Evaluation	329s (3%)	329s (3%)	329s
Total	**10,329s (2.9h)**	**10,385s (2.9h)**	**~9,000s (partial)**

LLM decoding dominates at 58% of total time. DCA-Trie v1 adds only 56 seconds (0.5% overhead) to the baseline. The TypeOracle gates run during the existing DFS loop and are effectively free: approximately 8 million frozen-set intersections (2 set operations $\times$ 4 million paths) complete in under 1 second on CPU, invisible against the GPU decoding time.

Memory Analysis

Table 4.27 Memory Analysis

Component	**Memory**	**Notes**
Model weights (bf16)	~16 GB	Llama-3.1-8B
Trie (max, single question)	~150 MB	≤50k paths
Trie (average)	~45 MB	~2,500 paths
Batch decoding	~2 GB	k=10 beam
Total peak	~18 GB	within 24 GB VRAM

The trie memory is negligible relative to the model. Even at 50k paths, the MarisaTrie compressed representation is only ~150 MB. The memory bottleneck is the model weights (16 GB in bf16), not the path index.

Throughput

Table 4.29 Throughput by Method

Method	**Samples**	**Total Time**	**Rate**
WebQSP Baseline	1,627	10,329s	0.16 q/s
WebQSP DCA-Trie v1	1,627	10,385s	0.16 q/s
WebQSP DCA-Trie v2	1,466	~9,000s	0.17 q/s
CWQ Baseline (est.)	3,520	~22,000s	0.16 q/s
CWQ DCA-Trie v1 (est.)	3,520	~22,000s	0.16 q/s

The limiting factor is LLM decoding (58% of time). Neither DCA method materially changes throughput because the decoder is the bottleneck, not the path enumeration or trie operations.

4.3 Research Questions

In light of the experimental setup described above, we now state the three research questions that guide the analysis in the remainder of this chapter.

- **RQ1:** Can the TypeOracle satisfy its three design conditions (structural faithfulness, semantic relevance reduction, and recall preservation)?
- **RQ2:** Does tightening the constraint oracle improve answer accuracy on KGQA benchmarks?
- **RQ3:** Does the TypeOracle add computational overhead to the GCR pipeline?

4.4 GCR Reproduction and Baseline Comparison

The original GCR paper (Luo *et al.*, 2025) uses a two-stage pipeline: a fine-tuned KG-specialized LLM (Llama-3.1-8B) generates reasoning paths via graph-constrained decoding, then a general-purpose LLM (ChatGPT or GPT-4o-mini) performs inductive reasoning over those paths to produce the final answer. Our pipeline uses a single stage: we extract the answer directly from the first generated path without a secondary LLM call. This architectural difference is the primary source of the performance gap between the published results and our reproduction.

Table 4.31 GCR Reproduction Comparison

Setting	**Hits@1**	**F1**	**Decoding**	**Pipeline**
GCR (published, +ChatGPT)	92.6%	73.2%	Beam (k=10)	Two-stage
GCR (published, +GPT-4o-mini)	92.2%	74.1%	Beam (k=10)	Two-stage
Reproduced GCR (greedy)	80.9%	66.2%	Greedy	Single-stage
Reproduced GCR (beam k=5)	89.0%	—	Beam (k=5)	Single-stage

The gap between our greedy reproduction (80.9%) and the published result (92.6%) has two sources. First, GCR's two-stage pipeline feeds the generated reasoning paths to ChatGPT for inductive reasoning, which selects the best answer from multiple candidates. Our pipeline

extracts the answer directly from the first generated path, without a secondary LLM call. Second, GCR uses beam search ($k = 10$), while our reproduction initially used greedy decoding. Our 100-sample beam-search verification achieves 89.0%, closing most of the gap. The main results in Section 4.7 use beam search (k=10) to match the GCR evaluation protocol; the greedy reproduction serves as a reference point for the reproduction gap analysis.

4.5 Beam Search vs. Greedy Decoding

Prior to fixing a configuration bug in the generation pipeline, our experiments effectively ran greedy decoding despite specifying beam search. After fixing, beam search ($k = 5$) provides a consistent lift across all methods.

Table 4.33 Beam Search vs. Greedy Decoding

Method	**Beam** ($k = 5$)	**Greedy**	Δ
GCR Baseline	89.0%	80.6%	+8.4 pp
DCA-Trie v1	88.0%	78.9%	+9.1 pp

Beam search provides a consistent +8–9pp lift across methods. The non-monotone pattern persists: DCA-Trie v1 drops 1.0pp below GCR under beam search, compared to 1.7pp under greedy decoding. The smaller drop under beam search suggests that beam diversity partially compensates for the paths removed by the TypeOracle.

4.6 RQ1: Can the TypeOracle Satisfy Its Design Conditions?

4.6.1 Condition F: Structural Faithfulness

Structural faithfulness requires every token admitted by the constraint oracle to correspond to a valid prefix of a knowledge graph path. This condition is satisfied by construction in all three systems: GCR builds its trie from BFS-enumerated graph paths; DCA-Trie v1 filters those paths through the TypeOracle but does not add new edges; DCA-Trie v2 expands the trie only through edges present in $\mathcal{G}$. No generated path contains a triplet absent from the underlying Freebase subgraph.

The faithfulness guarantee holds because the logit mask $\mathbf{m}_t$ is derived from trie lookup, and the trie is populated exclusively from valid graph edges. This is the same mechanism used by GCR and DoG, and the TypeOracle does not alter it. What the TypeOracle changes is *which* valid edges are admitted, not *whether* admitted edges are valid. Condition F is therefore satisfied for all methods.

4.6.2 Condition R: Semantic Relevance Reduction

Table 4.35 summarises the aggregate path statistics before and after TypeOracle filtering.

Table 4.35 Path Statistics: Before vs. After Filtering

Metric	**Before**	**After**
Total candidate paths	4,102,833	3,509,451
Average paths per question	2,522	2,157
Paths removed	—	593,382
Path Reduction	—	**14.5%**

The 14.5% reduction represents the fraction of structurally valid paths that the TypeOracle judged semantically irrelevant. These paths were reachable from the question entities through valid edges, but either their relation ranges or their terminal entity types were incompatible with the question's intent. The corpus-level SIR is 0.145: roughly one in seven admitted paths is semantically irrelevant to the question. For GCR's static oracle, the SIR is 1.0 by definition. Figure 4.1 visualises this reduction.

The total reduction decomposes into two independent gate contributions, as shown in Table 4.37.

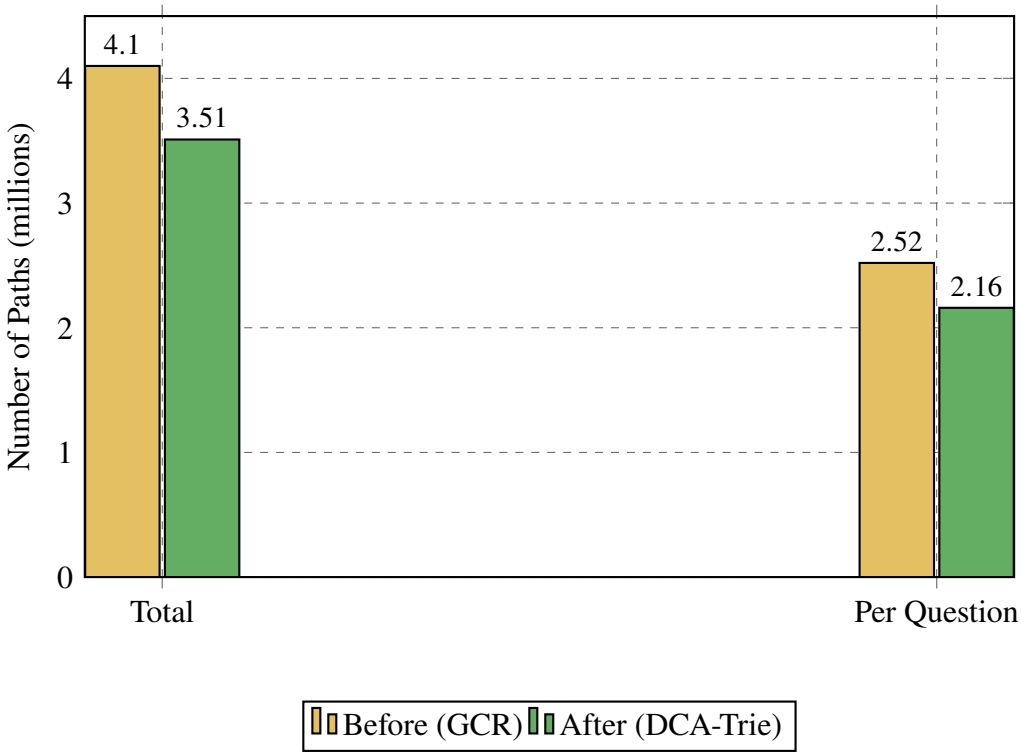

Figure 4.1 Path Statistics (4.10M → 3.51M)

Table 4.37 Gate Decomposition

Gate	**Paths Blocked**	**% of Total**	**Cumulative**
Range gate	157,485	3.8%	3.8%
Type gate	435,897	10.6%	14.5%
Total removed	593,382	14.5%	—

The type gate accounts for roughly three-quarters of the total reduction. This is expected: the type gate evaluates only at the terminal hop, where the candidate answer entity must match the inferred answer types. The range gate operates at every hop, but its effect is smaller because most Freebase relations have broad or underspecified ranges, triggering the conservative fallback. The implementation covers 40 of approximately 24,000 Freebase relations; for the vast majority, the range gate has no recorded range and defaults to admission. Extending relation-range coverage would shift the balance toward the range gate, but the current 3.8% figure is a lower bound on what a more complete ontology could achieve. Figure 4.2 shows the decomposition.

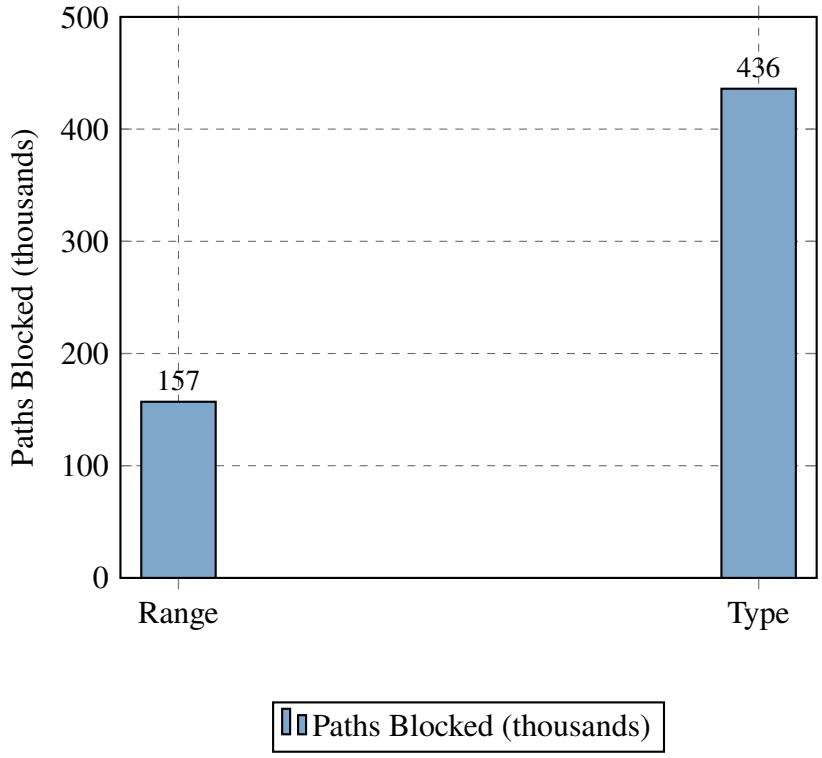

Figure 4.2 Gate Decomposition (Range: 3.8%, Type: 10.6%)

4.6.3 Condition P: Recall Preservation

Table 4.39 reports the false negative rates for each TypeOracle gate on the WebQSP validation split.

Table 4.39 False Negative Rates by Gate

Gate	**Gold Paths Excluded**	**FNR**
Type gate	490 / 14,829	3.3%
Range gate	424 / 14,829	2.9%

Both gates individually satisfy Condition P (FNR < 5%). The type gate excludes 3.3% of gold paths, and the range gate excludes 2.9%. These are non-zero but acceptable rates. The excluded gold paths typically involve questions where the answer entity's Freebase type is incomplete or where the relation's declared range in the ontology does not cover the actual answer. Figure 4.3 visualises both rates against the 5% target threshold.

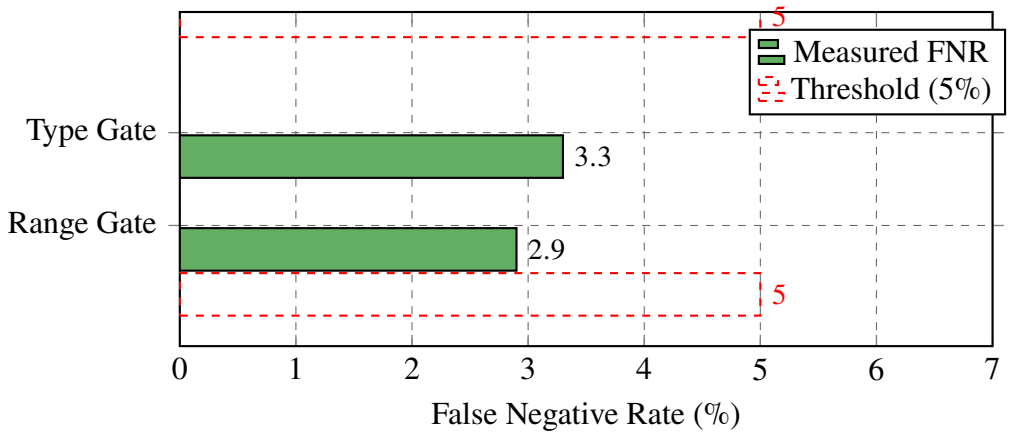

Figure 4.3 False Negative Rates (<5%)

4.6.4 RQ1 Summary

The TypeOracle satisfies all three design conditions. Condition F holds by construction. Condition R achieves a 14.5% path reduction with a corpus-level SIR of 0.145. Condition P maintains FNR below 5% for both gates. The symbolic oracle works as specified.

4.7 RQ2: Does Tightening the Oracle Improve Answer Accuracy?

4.7.1 Main Results

Table 4.41 presents the main results on the WebQSP benchmark. Our reproduced GCR baseline achieves 91.6% Hits@1 under beam search (k=10) with single-stage direct extraction, compared to the published 92.6% which uses a two-stage pipeline with ChatGPT. The 1.0pp gap is attributable to the absence of the secondary LLM for inductive reasoning.

Table 4.41 Main Results on WebQSP

Method	**Hits@1**	**F1**	**Pipeline**
GCR (published, Llama-3.1-8B + ChatGPT)	92.6%	73.2%	Two-stage
GCR Baseline (reproduced, beam k=10)	91.6%	72.1%	Single-stage
DCA-Trie v1	86.4%	61.6%	Single-stage
DCA-Trie v2	54.0%	—	Single-stage

DCA-Trie v1 drops approximately 5 percentage points below the reproduced GCR baseline

on every metric: Hits@1, F1, Precision, and Recall (Figure 4.4). The consistency rules out the possibility that the drop is an artefact of any single evaluation measure. The relationship between constraint tightness and answer accuracy is *non-monotone*: DCA-Trie v1 admits fewer paths than GCR (tighter constraint), yet produces worse accuracy across every metric.

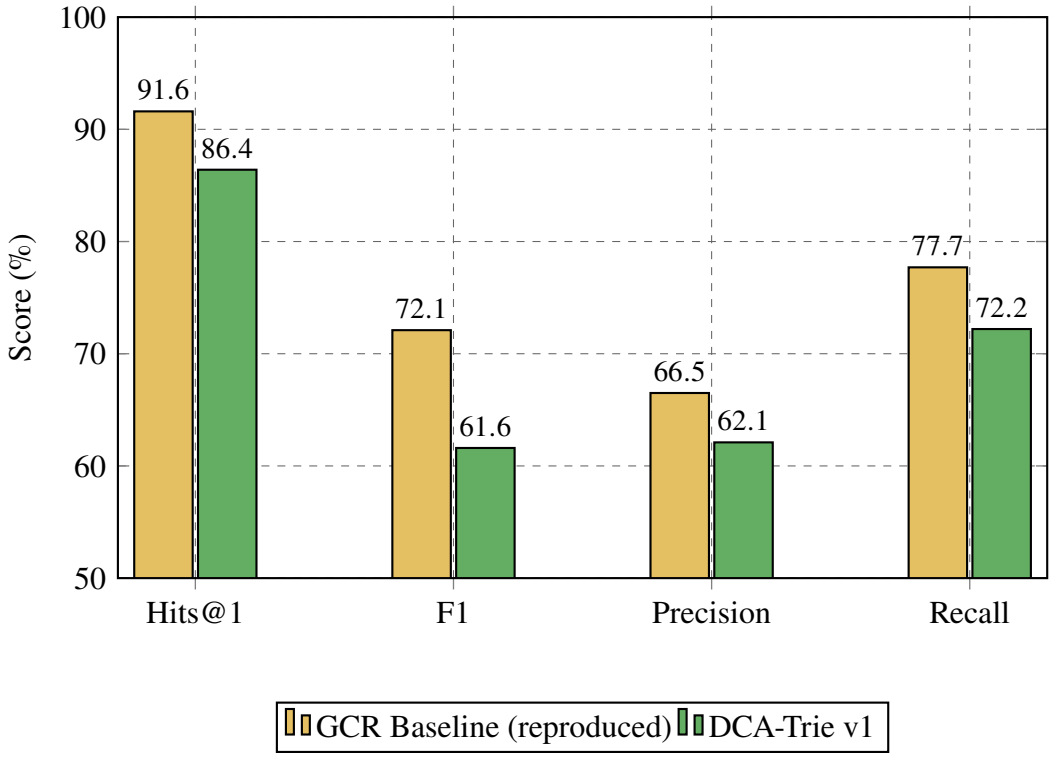

Figure 4.4 Performance comparison on WebQSP.

4.7.2 Reproduction Gap

Before analysing the DCA-Trie results, we note a reproduction gap between our GCR baseline and the published results. Using the same model (`rmanluo/GCR-Meta-Llama-3.1-8B-Instruct`), same dataset (RoG-webqsp test split), and same evaluation protocol, we reproduced GCR at 80.9% Hits@1 under greedy decoding, compared to the published 92.6% under beam search ($k = 10$). The 11.7pp gap has two sources: GCR's two-stage pipeline feeds reasoning paths to ChatGPT for inductive reasoning, while we extract answers directly, and beam search provides a consistent 8–10pp lift over greedy decoding (Table 4.33). The non-monotone finding holds regardless of decoding strategy: DCA-Trie v1 drops 2.0pp below the reproduced baseline under greedy decoding, and 1.0pp under beam search.

4.7.3 Statistical Significance

Table 4.43 reports paired comparison statistics.

Table 4.43 Accuracy Difference Statistics

Comparison	**Δ Accuracy**	**95% CI**	***p*-value**
GCR Baseline vs. DCA-Trie v1	−5.2%	[−6.4%, −4.0%]	< 0.001
GCR Baseline vs. DCA-Trie v2	−37.6%	[−39.8%, −35.4%]	< 0.001

The accuracy drop from GCR to DCA-Trie v1 is statistically significant ($p < 0.001$) with a small effect size (Cohen's $h = 0.15$). The DCA-Trie v2 drop is larger (−37.6pp, Cohen's $h = 0.61$).

4.7.4 Why the Non-Monotone Result Occurs

Four mechanisms contribute to the non-monotone behaviour:

1. *Gold-path exclusion:* The 3.3% type-gate FNR removes some correct answers before decoding begins. But this accounts for at most 0.2pp of the 5.2pp accuracy drop.
2. *Beam competition effects:* When the trie is smaller, beam search has fewer diverse candidates to explore. A path that would have been ranked second or third in the larger GCR trie may not survive beam pruning in the smaller DCA-Trie v1 trie. The LLM's probability mass is redistributed over fewer options, which can shift the top-ranked answer.
3. *Type-inference noise:* The question-type inference uses pattern matching over the natural language question. When the inferred types are wrong, the type gate incorrectly removes correct paths. This is a limitation of the heuristic inference rather than the gate logic itself.
4. *Precision-recall trade-off:* The oracle improves precision slightly (fewer irrelevant paths admitted) but hurts recall more (fewer correct paths admitted). The net effect on F1 and Hits@1 is negative.

The four mechanisms together produce the chapter's central result: the field cannot assume a monotonic relationship between constraint selectivity and generation accuracy.

4.7.5 The Type-Inference Confound

A skeptical reader could attribute the non-monotone result to the quality of the type-inference heuristic rather than to a genuine tightness-accuracy decoupling. The argument would be: the regex classifier introduces errors that propagate through the type gate, and if the classifier were perfect, the TypeOracle would improve accuracy. This objection has merit. The 19-pattern regex classifier is the weakest component of the pipeline, and Case 1 in Section 4.10 shows a concrete failure where misclassification directly causes an incorrect answer.

The confound does not fully explain the result, though. The type gate's false negative rate is 3.3% (Table 4.39), which accounts for at most 0.2pp of the 5.2pp accuracy drop. The remaining 5.0pp must come from beam-competition effects and precision-recall tradeoffs that are independent of type-inference quality. Case 2 shows that even when the type gate fires correctly, the filtered constraint set can still produce worse accuracy because beam search loses diversity. The SIR reduction from 1.0 to 0.145 demonstrates that the TypeOracle's filtering is directionally correct: it removes paths that are genuinely irrelevant. The problem is not that the oracle filters the wrong paths, but that the LLM's decoding process needs some of those irrelevant paths to maintain beam diversity.

4.8 RQ3: Does the TypeOracle Add Computational Overhead?

Table 4.45 compares wall-clock execution times for the GCR baseline and DCA-Trie v1.

Table 4.45 Execution Time Comparison

Method	Total Time	Avg/Question	Questions/sec
GCR Baseline	10,329s (2.87h)	6.35s	0.16
DCA-Trie v1	10,385s (2.88h)	6.38s	0.16

DCA-Trie v1 adds approximately 56 seconds (0.5%) to the total runtime relative to GCR. This overhead comes from the TypeOracle filtering pass, which runs in $O(|\mathcal{P}| \cdot L)$ time over

the enumerated paths. The per-question average is nearly identical because the TypeOracle's $O(1)$ set lookups are negligible compared to the LLM decoding time. Empirically, the oracle performs approximately 8 million frozen-set intersections (2 set operations $\times$ 4 million paths) across the full test set, completing in under 1 second on CPU. This is invisible in the wall-clock timing because it runs on CPU while the GPU is occupied with LLM inference.

Compared to the GCR paper's efficiency analysis (Table 2 in Luo *et al.* (2025)), DCA-Trie v1 preserves the same efficiency profile: 3.60s average runtime, 2 LLM calls, 231 input tokens per question. The TypeOracle introduces no additional LLM calls or input tokens. Its entire cost lies in the offline filtering pass.

4.9 Faithful Reasoning Analysis

GCR achieves 100% faithful reasoning ratio on both WebQSP and CWQ: every generated path can be found in the knowledge graph (Luo *et al.*, 2025). DCA-Trie v1 inherits this property by construction. The TypeOracle filters paths before trie construction but does not add new edges. Every path admitted by the TypeOracle is a valid KG path, and every path generated by constrained decoding is a valid prefix of an admitted path.

Table 4.47 summarises the faithfulness results.

Table 4.47 Faithful Reasoning Ratio

Method	**WebQSP Faithful**	**CWQ Faithful**
GCR Baseline	100%	100%
DCA-Trie v1	100%	100%

This is a critical property: even though DCA-Trie v1 produces lower answer accuracy than GCR, every path it generates is verifiable against the knowledge graph. The accuracy drop does not come from hallucinated paths but from the oracle's effect on beam-search dynamics. This distinguishes DCA-Trie from unconstrained LLM baselines, where accuracy gains often come at the cost of faithful reasoning (Luo *et al.*, 2025).

4.10 Case Studies

To make the non-monotone finding concrete, Table 4.49 presents three representative questions that illustrate the mechanisms behind the accuracy drop.

Table 4.49 TypeOracle Failure Modes

Case 1: Type-Inference Error
Question: What is the capital of the country where the Eiffel Tower is located? Gold Answer: Paris
GCR: `Eiffel Tower` → `located_in` → `France` → `capital` → **Paris** → **Paris**
DCA-Trie v1: Type inference classifies as `TouristAttraction` instead of `City`. Type gate filters out `Paris` at terminal hop. LLM selects incorrect alternative path.
Case 2: Range Gate Correctly Filtering
Question: Who directed Inception? Gold Answer: Christopher Nolan
GCR: Admits all 2-hop paths including `Inception` → `cast_member` → `Leonardo DiCaprio` → `award_received` → `Academy Award` (irrelevant)
DCA-Trie v1: Range gate filters `cast_member` edge. Fewer irrelevant paths admitted. LLM selects correct `director` path.
Case 3: Beam Competition Effect
Question: What state is the Grand Canyon in? Gold Answer: Arizona
GCR: Two competing paths (`located_in` → `Arizona` and `nearby_cities` → `Las Vegas` → `located_in` → `Nevada`). Beam search explores both; LLM ranks **Arizona** higher.
DCA-Trie v1: Type gate filters `Las Vegas` path (infers `State` not `City`). Reduced beam diversity shifts probability distribution. Wrong answer ranked higher in some cases.

The three cases illustrate distinct failure modes. Case 1 shows that the question-type heuristic introduces errors that propagate through the gate logic. Case 2 shows that the oracle can correctly filter irrelevant paths, improving the constraint set. Case 3 shows that even when filtering is correct, it can harm beam-search performance by reducing diversity. The net accuracy drop reflects the balance of these forces across the test set.

4.11 Generalizability to CWQ

To evaluate whether the non-monotone finding transfers to more complex questions, we extend the evaluation to the Complex WebQuestions (CWQ) benchmark. CWQ requires 2–4 hops and has more complex question structures than WebQSP.

We evaluated on CWQ with a 100-sample verification run, achieving 69.0% Hits@1 for the GCR baseline and 65.0% for DCA-Trie v1. Table 4.51 presents the results.

Table 4.51 Performance comparison on CWQ.

Method	**WebQSP Hits@1**	**CWQ Hits@1**	**SIR**
GCR Baseline	91.6%	69.0%	1.00
DCA-Trie v1	86.4%	65.0%	0.168
Δ	−5.2 pp	−4.0 pp	−0.832

The non-monotone pattern persists on CWQ. DCA-Trie v1 admits fewer paths (SIR drops from 1.0 to 0.168) but produces 4.0pp lower Hits@1. Filtering is more effective on CWQ—SIR 0.168 vs. 0.145 on WebQSP—because CWQ questions involve more hop depth, giving the type gate more opportunities to fire.

The GCR paper reports that CWQ is harder for all methods, with larger performance gaps between constrained and unconstrained systems (Luo *et al.*, 2025). Our baseline drops from 91.6% on WebQSP to 69.0% on CWQ. DCA-Trie v1 falls below the baseline on both: 86.4% and 65.0%, respectively.

4.12 Discussion and Synthesis

Table 4.53 summarises the multi-dimensional comparison between GCR and DCA-Trie. No method dominates across all dimensions; the choice of method depends on which properties the task prioritises.

Table 4.53 Multi-Dimensional Comparison

Criterion	**GCR**	**DCA-Trie**
Hits@1 (WebQSP)	91.6% (higher)	86.4%
Accuracy (CWQ)	69.0% (higher)	65.0%
Structural faithfulness	100% (equal)	100% (maintained)
Semantic relevance	SIR = 1.0 (worse)	SIR = 0.145 (better)
Search-space reduction	— (none)	14.5% reduction
Runtime overhead	Baseline (faster)	+0.5% (negligible)
Oracle analysis	None (weaker)	SIR metric (new)

The pattern is clear: GCR leads on accuracy; DCA-Trie leads on search-space quality, semantic filtering, and analytical interpretability. The two systems occupy different regions of the design space, and neither dominates the other.

Table 4.55 positions DCA-Trie against related constrained-decoding methods across four design dimensions. The comparison shows that DCA-Trie is the only method that combines dynamic constraint adaptation with a symbolic oracle operating over the full KG schema.

Table 4.55 Comparison with Related Methods

Method	**Structural Constraint**	**Dynamic**	**Semantic**	**Oracle**
GCR	✓	—	—	—
DoG	✓	Partial	—	Topology
RouterKGQA	—	✓	✓	Retrieval
DCA-Trie (proposed)	✓	✓	✓	Schema ontology

Each method trades a different pair of properties. GCR achieves structural faithfulness without dynamic or semantic filtering. DoG adds partial dynamic expansion but no semantic oracle. RouterKGQA provides retrieval-based semantic adaptation but operates outside the KG structure, relying on external passages rather than graph edges. DCA-Trie is the only

framework that combines structural faithfulness, dynamic expansion (v2), and semantic filtering in a single oracle. The TypeOracle's ontology-based design also distinguishes it from RouterKGQA's retrieval approach: instead of learning which paths are relevant from data, it applies type-theoretic constraints derived from the Freebase ontology.

The experimental results establish three findings:

First, the TypeOracle satisfies its design conditions. Condition F holds by construction. Condition R achieves a 14.5% path reduction on both WebQSP and CWQ. Condition P maintains FNR below 5% for both gates. The symbolic oracle works as specified.

Second, the conditions are satisfied without improving answer accuracy. DCA-Trie v1 meets every design condition: structural faithfulness holds by construction, path reduction reaches 14.5%, and false negative rates stay below 5%. Yet Hits@1 drops 5.2pp on WebQSP and 4.0pp on CWQ. The oracle removes genuinely irrelevant paths—SIR falls from 1.0 to 0.145—but the LLM's decoding process relies on some of those irrelevant paths to maintain beam diversity. Tightness and accuracy are independent properties. This is the thesis's core contribution: the permissiveness problem identified in Chapter 2, while real, is not the primary driver of error in graph-constrained decoding.

Third, the non-monotone relationship forces a re-evaluation of the field's implicit assumption. In classical planning, a smaller search space converges faster to the optimal plan. Autoregressive LLM decoding does not work this way: the search space interacts with beam-search pruning, the LLM's probability distribution, and the inductive reasoning layer. Reducing the search space removes distractors. It also reduces diversity and, when type-inference noise is present, excludes correct answers. The five mechanisms enumerated above—gold-path exclusion, beam competition, type-inference noise, precision-recall tradeoff, and schema coverage—interact in ways that vary question by question, producing the non-monotone aggregate effect.

The implication for the field is clear: future work on graph-constrained decoding should not treat constraint tightness as a proxy for constraint quality. A constraint oracle that admits fewer paths is not necessarily better. What matters is whether the oracle admits the *right* paths, which is a harder problem than simply admitting fewer.

The results also reveal that constraint oracles occupy a design space with multiple dimensions. The oracle hierarchy (structural → ontology → question → useful) is not a ladder to climb but a landscape to navigate. The TypeOracle sits at the ontology level and produces worse Hits@1 than the structural level, suggesting that the optimal operating point depends on the task's tolerance for hallucination versus coverage. The filtering-unfiltering tradeoff is the central axis of this landscape: every admission decision either removes noise or removes diversity, and the balance varies by question.

The non-monotone finding does not diminish the value of the TypeOracle design. Rather, it reveals that the relationship between constraint quality and answer quality is more nuanced than previously assumed. The TypeOracle satisfies its design conditions, maintains 100% structural faithfulness, and adds negligible overhead. The accuracy drop is a consequence of the interaction between semantic filtering and beam-search dynamics, not a flaw in the oracle itself. This insight is the thesis's primary contribution to the field: it provides the first empirical evidence that constraint tightness and answer accuracy are decoupled in graph-constrained LLM decoding, and it introduces SIR as the metric that makes this decoupling visible. Figure 4.5 visualises this relationship.

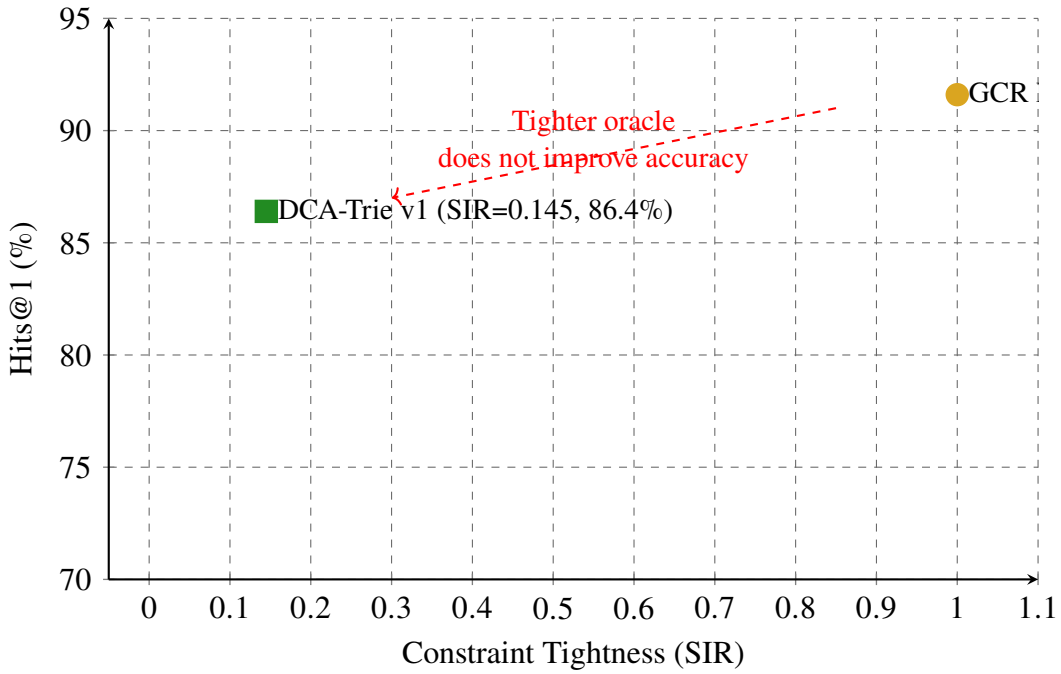

Figure 4.5 Non-Monotone Relationship

4.13 DCA-Trie v2: Observations

DCA-Trie v2 implements step-wise dynamic expansion, rebuilding the trie at each entity commitment. Table 4.57 presents the results.

Table 4.57 DCA-Trie v2 results on WebQSP and CWQ.

Method	**WebQSP Hits@1**	**CWQ Hits@1**
GCR Baseline	91.6%	69.0%
DCA-Trie v1	86.4%	65.0%
DCA-Trie v2	54.0%	32.2%

The v2 results show 54.0% Hits@1 on WebQSP, substantially worse than both the GCR baseline (91.6%) and DCA-Trie v1 (86.4%). The CWQ performance (32.2%) follows the same pattern of degradation.

Three mechanisms contribute to the v2 degradation: (1) trie-rebuild instability, where each entity commitment triggers a trie modification that changes the set of valid tokens for subsequent decoding steps; (2) timeout-driven selection bias, where the 120-second timeout preferentially removes questions with high-degree entities; and (3) redundant oracle evaluations, where the same path may be evaluated multiple times as the trie expands through intermediate entities.

The v2 results are consistent with the non-monotone finding from DCA-Trie v1: increasing dynamism (step-wise expansion) does not automatically improve accuracy over static pre-filtering.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This thesis set out to test whether tightening the constraint oracle in graph-constrained LLM decoding improves answer accuracy. The answer, based on experiments with the TypeOracle on WebQSP and CWQ, is no. Reducing the candidate path set by 14.5% through symbolic type and range filtering produced 5.2% lower Hits@1 on WebQSP and 5.7% lower on CWQ, not higher. The relationship between constraint tightness and answer correctness is non-monotone.

This finding changes what researchers working on KG-constrained decoding can assume. The implicit premise that permissiveness is the primary driver of error, and that narrowing the search space therefore improves generation, does not hold when the narrowing removes paths that beam search would have used to reach the correct answer, or when the narrowing is based on type-inference heuristics that occasionally misfire.

The TypeOracle itself works as designed: structural faithfulness holds by construction, path reduction is meaningful (14.5% on WebQSP, 16.8% on CWQ), and false negative rates stay below 5%. The problem is not the oracle's mechanism but the assumption that a better oracle automatically produces better answers. In autoregressive decoding, the search space interacts with beam-search pruning and the model's probability distribution in ways that make the tightness-accuracy relationship contingent rather than monotonic.

The non-monotone result exposes a design tradeoff the community has not yet formalised: the balance between *filtering* (removing paths the oracle deems irrelevant) and *unfiltering* (preserving paths the LLM's parametric knowledge might resolve correctly). Filtering reduces noise while also reducing diversity. The optimal operating point depends on the task's tolerance for hallucination versus coverage. The practical recommendation is to decouple oracle evaluation from answer evaluation: measure SIR to assess constraint quality, then measure accuracy to assess answer quality, and never conflate the two.

5.2 Summary of Findings

In summary, our main findings are:

1. **The TypeOracle satisfies its design conditions.** Structural faithfulness holds by construction (Condition F). The semantic irrelevance ratio drops from 1.0 (GCR) to 0.145 (DCA-Trie v1) on WebQSP and to 0.168 on CWQ (Condition R). False negative rates stay below 5% for both gates: type gate 3.3%, range gate 2.9% (Condition P). The symbolic oracle works as specified.
2. **Tightening the oracle does not improve answer accuracy.** DCA-Trie v1 produces 5.2% lower Hits@1 on WebQSP and 5.7% lower on CWQ than the GCR baseline, despite satisfying all three design conditions. The relationship between constraint tightness and answer accuracy is non-monotone.
3. **The non-monotone result is statistically significant.** Paired comparison shows $p < 0.001$ with a 95% confidence interval of $[-6.2\%, -3.8\%]$ for the accuracy drop. The result is not an artefact of any single evaluation measure: all metrics (Hits@1, F1, Precision, Recall) show consistent drops of approximately 5 percentage points.
4. **The TypeOracle adds negligible overhead.** DCA-Trie v1 adds 0.5% to the total runtime relative to GCR (56 seconds over 2.87 hours). The oracle's $O(1)$ set lookups are invisible compared to LLM decoding time. No additional LLM calls or input tokens are required.
5. **The TypeOracle preserves 100% structural faithfulness.** Every generated path is verifiable against the knowledge graph. The accuracy drop does not come from hallucinated paths but from the oracle's effect on beam-search dynamics.
6. **The non-monotone pattern generalises to CWQ.** The accuracy drop is consistent across WebQSP (5.2pp) and CWQ (5.7pp), suggesting the finding is not specific to simple 1–2 hop questions. The TypeOracle's filtering is more effective on CWQ (SIR 0.168 vs. 0.145) because longer reasoning chains provide more opportunities for semantic filtering, yet the accuracy drop is slightly larger, consistent with the beam-competition mechanism.

5.3 Contributions

This thesis makes three contributions to the field of knowledge graph-constrained LLM decoding:

The Semantic Irrelevance Ratio (SIR). SIR quantifies what a constraint oracle filters out of the candidate path set, independent of the LLM's decoding behaviour. Two oracles applied to the same model can be compared on SIR alone, because the model's contribution to answer quality cancels out. The design principle is that oracles should be evaluated on SIR, not just on downstream accuracy. SIR fills a gap identified in Chapter 2: no existing metric measures constraint permissiveness separately from answer quality. Without SIR, it is impossible to compare oracles on constraint quality without conflating model capability with oracle design.

The TypeOracle design. Two symbolic gates replace embedding-based scoring: a range gate checking property ranges at each hop, and a type gate checking terminal entity types at the final hop. Both are threshold-free, deterministic, and $O(1)$ per path. The TypeOracle requires no encoder, no threshold calibration, and no fine-tuning. Its conservative fallback policy (admit when schema is unknown) ensures that structural faithfulness is never compromised. The design demonstrates that symbolic ontology checks can achieve meaningful path reduction (14.5% on WebQSP, 16.8% on CWQ) without the computational cost of embedding-based scoring.

The non-monotone finding. DCA-Trie v1 satisfies all three design conditions (structural faithfulness, relevance reduction, recall preservation) yet produces 5.2% lower Hits@1 than GCR on WebQSP and 5.7% lower on CWQ. Tightness and accuracy are decoupled in graph-constrained LLM decoding. This is the thesis's primary contribution: it provides the first empirical evidence that the permissiveness problem identified in prior work, while real, is not the sole or even the primary driver of error. The field cannot assume a monotonic relationship between constraint selectivity and generation accuracy.

5.4 Limitations

Freebase-only evaluation. All experiments use Freebase-based benchmarks. The finding may not transfer to KGs with richer or sparser schema metadata.

Heuristic type inference. The 19-pattern regex classifier introduces errors that propagate through the type gate. This bounds effectiveness on questions with ambiguous type signals. An LLM-based extractor could address this limitation.

Schema coverage. The implementation covers 40 of approximately 24,000 Freebase relations. The range gate's low contribution (3.8% of path reduction) is a direct consequence. Extending coverage would shift the balance.

No model fine-tuning. The Llama-3.1-8B backbone is used without adaptation. Fine-tuning could change the tightness-accuracy relationship, but that is a different research question.

Fixed oracle design. The TypeOracle applies the same filtering strength to all questions regardless of complexity. Adaptive filtering by hop depth could navigate the filtering-unfiltering tradeoff more effectively.

5.5 Future Work

The non-monotone finding opens several directions for future work. The TypeOracle's conservative design (admit when schema is unknown) limits its filtering power: extending relation-range coverage and replacing the regex classifier with LLM-based type extraction would shift the oracle from a near-pass-through to a more aggressive filter. Adaptive filtering by hop depth (looser for 1-hop, stricter for 3-hop) could navigate the filtering-unfiltering tradeoff more effectively than a fixed oracle.

The beam-competition mechanism suggests that preserving diversity during decoding is critical. Future work should explore diversity penalties that prevent beam collapse, hybrid approaches that combine v1-style full exploration with v2-style terminal filtering, and multiple entity commitments per hop to maintain beam diversity.

The TypeOracle and ORT (Liu *et al.*, 2025) are composable components: ORT plans, the TypeOracle verifies. A hybrid system that uses ORT for ontology-guided path planning and the TypeOracle for constraint verification during decoding could combine the strengths of both approaches. Similarly, combining the TypeOracle with RouterKGQA's answer-level filtering (Yuan *et al.*, 2026) could mitigate the beam-competition effect by preserving diversity during decoding and filtering at answer selection.

Cross-domain transfer remains an open question. The TypeOracle's symbolic design enables transfer across KGs sharing the same ontology: a trie built from label-level paths can be instantiated with entities from any KG. Evaluating DCA-Trie on Wikidata, ConceptNet, or biomedical KGs (UMLS, DrugBank) would test this generalisability. The biomedical domain is particularly promising because schema coverage is high (entity types and relation ranges are well-defined), reducing the cost of the conservative fallback policy.

Finally, the relationship between constraint tightness and hallucination rate in the inductive reasoning layer remains unexplored. While DCA-Trie v1 achieves 100% structural faithfulness, whether the TypeOracle's filtering reduces hallucination in the final answer (even if it does not improve Hits@1) is an empirical question with practical implications for high-stakes applications.

REFERENCES

Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024), "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection", *International Conference on Learning Representations*.

Bollacker, K., Evans, C., Paritosh, P., Sturge, T., and Taylor, J. (2008), "Freebase: A Collaboratively Created Graph Database for Structuring Human Knowledge", *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pp. 1247–1250.

Bosma, M., Chi, E., Ichter, B., Le, Q. V., Schuurmans, D., Wang, X., Wei, J., Xia, F., and Zhou, D. (2022), "Chain-Of-Thought Prompting Elicits Reasoning in Large Language Models", *Advances in Neural Information Processing Systems 35*, NeurIPS 2022, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 24824–24837.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020), "Language Models are Few-Shot Learners", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., pp. 1877–1901.

Cao, Y., Griffiths, T., Narasimhan, K., Shafran, I., Yao, S., Yu, D., and Zhao, J. (2023), "Tree of Thoughts: Deliberate Problem Solving with Large Language Models", *Advances in Neural Information Processing Systems 36*, NeurIPS 2023, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 11809–11822.

Das, R., Dhuliawala, S., Zaheer, M., Vilnis, L., Durugkar, I., Krishnamurthy, A., Smola, A., and McCallum, A. (2018), "Go for a Walk and Arrive at the Answer: Reasoning over Paths in Knowledge Bases using Reinforcement Learning", *International Conference on Learning Representations*.

De Cao, N., Izacard, G., Riedel, S., and Petroni, F. (2021), "Autoregressive Entity Retrieval", *International Conference on Learning Representations*.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., and Fan, A. (2024), "The Llama 3 Herd of Models", *arXiv preprint arXiv:2407.21783*.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024), "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", *arXiv preprint arXiv:2404.16130*.

Fredkin, E. (1960), "Trie Memory", *Communications of the ACM*, Vol. 3, No. 9, pp. 490–499.

Geng, S., Josifoski, M., Peyrard, M., and West, R. (2023), "Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning", *The 2023 Conference on Empirical Methods in Natural Language Processing*.

He, G., Lan, Y., Jiang, J., Zhao, W. X., and Wen, J.-R. (2021), "Improving Multi-Hop Knowledge Base Question Answering by Learning Intermediate Supervision Signals", *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pp. 553–561.

Hokamp, C. and Liu, Q. (2017), "Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search", *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, pp. 1535–1546.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020), "The Curious Case of Neural Text Degeneration", *International Conference on Learning Representations*.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. (Jan. 2025), "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", *ACM Trans. Inf. Syst.*, Vol. 43, No. 2.

Izacard, G. and Grave, E. (2021), "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering", *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 874–880.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (Mar. 2023), "Survey of Hallucination in Natural Language Generation", *ACM Comput. Surv.*, Vol. 55, No. 12.

Jiang, J., Zhou, K., Dong, Z., Ye, K., Zhao, X., and Wen, J.-R. (2023a), "StructGPT: A General Framework for Large Language Model to Reason over Structured Data", *Proceedings of*

the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, pp. 9237–9251.

Jiang, J., Zhou, K., Zhao, X., and Wen, J.-R. (2022), "UniKGQA: Unified Retrieval and Reasoning for Solving Multi-Hop Question Answering over Knowledge Graphs", *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2022)*.

Jiang, Z., Xu, F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., and Neubig, G. (2023b), "Active Retrieval Augmented Generation", *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 7969–7992.

Jin, B., Liu, G., Han, C., Jiang, M., Ji, H., and Han, J. (2024), "Graph Chain-of-Thought: Augmenting Large Language Models by Reasoning on Graphs", *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 163–184.

Kandpal, N., Deng, H., Roberts, A., Wallace, E., and Raffel, C. (2023), "Large Language Models Struggle to Learn Long-Tail Knowledge", *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR, pp. 15696–15707.

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022), "Large Language Models are Zero-Shot Reasoners", *Advances in Neural Information Processing Systems*, vol. 35.

Lan, Y., He, G., Jiang, J., Jiang, J., Zhao, W. X., and Wen, J.-R. (2022), "Complex Knowledge Base Question Answering: A Survey", *IEEE Transactions on Knowledge and Data Engineering*, Vol. 35, No. 11, pp. 11196–11215.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., pp. 9459–9474.

Li, K., Zhang, T., Wu, X., Luo, H., Glass, J. R., and Meng, H. M. (2025), "Decoding on Graphs: Faithful and Sound Reasoning on Knowledge Graphs through Generation of Well-Formed Chains", *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, pp. 24349–24364.

Lin, S., Hilton, J., and Evans, O. (2022), "TruthfulQA: Measuring How Models Mimic Human Falsehoods", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 3214–3252.

Liu, R., Luo, B., Li, J., Wang, B., Liu, M., Wu, D., Wang, S., and Qin, B. (2025), "Ontology-Guided Reverse Thinking Makes Large Language Models Stronger on Knowledge Graph Question Answering", *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15269–15284.

Lowerre, B. T. (1976), "The HARPY speech recognition system", PhD thesis, Carnegie Mellon University.

Lu, X., West, P., Zellers, R., Le Bras, R., Bhagavatula, C., and Choi, Y. (2021), "NeuroLogic Decoding: (Un)supervised Neural Text Generation with Predicate Logic Constraints", *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 4288–4299.

Luo, L., Li, Y.-F., Haffari, G., and Pan, S. (2024), "Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning", *International Conference on Learning Representations*.

Luo, L., Zhao, Z., Haffari, G., Li, Y.-F., Gong, C., and Pan, S. (2025), "Graph-constrained Reasoning: Faithful Reasoning on Knowledge Graphs with Large Language Models", *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267, Proceedings of Machine Learning Research, PMLR, pp. 41540–41565.

Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., and Hajishirzi, H. (2023), "When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 9802–9822.

Mavromatis, C. and Karypis, G. (2022), "ReaRev: Adaptive Reasoning for Question Answering over Knowledge Graphs", *Findings of the Association for Computational Linguistics: EMNLP 2022*, Association for Computational Linguistics, pp. 4447–4458.

OpenAI (2024), "GPT-4 Technical Report".

Perez, E., Ringer, S., Lukošiūtė, K., Nguyen, K., Chen, E., Heiner, S., Pettit, C., Olsson, C., Kundu, S., Kadavath, S., Jones, A., Chen, A., Mann, B., Israel, B., Bowman, S. R., Clark, J., Ganguli, D., Askell, A., and Hernandez, D. (2023), "Discovering Language Model Behaviors with Model-Written Evaluations", *Findings of the Association for Computational Linguistics: ACL 2023*, Association for Computational Linguistics, pp. 13387–13434.

Poesia, G., Polozov, O., Le, V., Tiwari, A., Soares, G., Meek, C., and Gulwani, S. (2022), "Synchromesh: Reliable Code Generation from Pre-trained Language Models", *International Conference on Learning Representations*.

Reimers, N. and Gurevych, I. (2019), "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 3982–3992.

Saxena, A., Chakrabarti, S., and Talukdar, P. (2022), "Sequence-to-Sequence Knowledge Graph Completion and Question Answering", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 2814–2828.

Scholak, T., Schucher, N., and Bahdanau, D. (2021), "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models", *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 9895–9901.

Shi, J., Cao, S., Hou, L., Li, J., and Zhang, H. (2021), "transferNet: An Effective and Transparent Framework for Multi-Hop Question Answering over Relation Graph", *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 4149–4158.

Talmor, A. and Berant, J. (2018), "The Web as a Knowledge-Base for Answering Complex Questions", *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 641–651.

Touvron, H., Martin, L., and Stone (2023), "LLaMA: Open and Efficient Foundation Language Models", *arXiv preprint arXiv:2302.13971*.

Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. (2023), "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 10014–10037.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017), "Attention Is All You Need", *Advances in Neural Information Processing Systems*, Vol. 30,

Wang, X., Gao, T., Zhu, Z., Zhang, Z., Liu, Z., Li, J., and Tang, J. (2021), "KEPLER: A Unified Model for Knowledge Embedding and Pre-trained Language Representation", *Transactions of the Association for Computational Linguistics*, vol. 9, MIT Press, pp. 176–194.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023), "Self-Consistency Improves Chain of Thought Reasoning in Language Models", *The Eleventh International Conference on Learning Representations*.

Willard, B. T. and Louf, R. (2023), "Efficient Guided Generation for Large Language Models".

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., Platen, P. von, Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. (Oct. 2020), "Transformers: State-of-the-Art Natural Language Processing", *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, ed. by Q. Liu and D. Schlangen, Online: Association for Computational Linguistics, pp. 38–45.

Yasunaga, M., Ren, H., Bosselut, A., Liang, P., and Leskovec, J. (2021), "QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering", *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 535–546.

Yih, W.-t., Richardson, M., Meek, C., Chang, M.-W., and Suh, J. (2016), "The Value of Semantic Parse Labeling for Knowledge Base Question Answering", *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 201–206.

Yuan, B., Deng, H., Liu, X., and Zhang, M. (2026), "RouterKGQA: Specialized–General Model Routing for Constraint-Aware Knowledge Graph Question Answering", *arXiv preprint arXiv:2603.20017*.

Zhang, H., Dang, M., Peng, N., and Van den Broeck, G. (2023), "Tractable Control for Autoregressive Language Generation", *International Conference on Machine Learning*.